HedgeSwap: Universal Hedged Atomic Swaps Against Griefing Attacks

Dongkun Hou[†], Ying-Teng Chen[†], Shuijie Cui[†], Tsz Hon Yuen[†], Joseph K. Liu[†], and Jiangshan Yu[‡]
[†]Monash University, Clayton, VIC, Australia
{dongkun.hou, ying-teng.chen, shujie.cui, john.tszhonyuen, joseph.liu}@monash.edu
[‡]The University of Sydney, Sydney, NSW, Australia
jiangshan.yu@sydney.edu.au

***Abstract*—Universal atomic swaps [Oakland'22] replace hashed timelock contracts with adaptor signatures and verifiable timed dlogs, enabling secure cross-chain cryptocurrency exchanges that only require basic signature verification from the underlying blockchains. However, existing universal atomic swap protocols remain vulnerable to *griefing attacks*, where a deviating party aborts the swap to lock a compliant party's assets for a long period. A natural approach is to lift existing contract-based solutions to the universal setting, but we identify that this straightforward solution faces two key challenges: (i) *timeout race attacks*, first identified in PipeSwap [Oakland'25], which arises from the absence of an upper bound on the transaction validity; (ii) a *timeout overlap dilemma*, which results from multiple overlapping refund periods.**

In this paper, we propose *HedgeSwap*, a universal hedged atomic swap protocol against griefing attacks, which compensates a compliant party with a premium if its asset is locked but not redeemed. To mitigate the timeout race attacks and timeout overlap dilemma, HedgeSwap eliminates the premium timeout and instead relies on a hard relation to refund the premium. For high-value asset swaps where the parties acceptable premium ranges do not overlap, we further propose a *round-based HedgeSwap* that utilizes a *premium migration mechanism* to solve these two timeout challenges, where parties iteratively increase the premium until the lock-up risk premium acceptable to both. Our experimental results show that our HedgeSwap can complete in under 0.5 seconds, and round-based HedgeSwap completes in under 1.3 seconds for a five-round setting, while HedgeSwap reduces gas cost by $2.69\times$ compared to existing contract-based solutions.

I. Introduction

Atomic swaps are crucial protocols for blockchain interoperability, enabling secure and trustless asset exchange across distinct blockchains [1], [2]. Atomicity guarantees that either all parties receive the expected assets or all retain their original holdings. A widely adopted method for achieving atomic swaps is the Hashed Timelock Contract (HTLC) [3].

HTLC-based protocols utilize a *hashlock* and a *timelock* script to enable conditional asset transfers based on either a correct preimage of a hash or a specified timeout. Roughly speaking, suppose Alice and Bob wish to exchange assets v_0 and v_1 on blockchain B_0 and B_1. Alice first selects a secret x, computes its hash $h = H(x)$, sets a timeout T_0, and locks v_0 into an HTLC on B_0. Bob then sets his timeout $T_1 > T_0$, and locks his v_1 into an HTLC using h on B_1. HTLC-based protocols ensure that if Alice redeems Bob's v_1 using x, Bob can then use x to redeem Alice's v_0; otherwise, both parties reclaim their original assets after the timeouts. Unfortunately, HTLC-based protocols rely on the scripting capability of the underlying blockchains, which are incompatible with scriptless blockchains such as ZCash [4], Monero [5], and Ripple [6].

Thyagarajan et al. [7] proposed the first universal atomic swap protocol, eliminating the need for HTLC scripts and requiring only signature verification from the underlying blockchains. This protocol replaces the hashlock and timelock scripts with *adaptor signatures* [8] and *verifiable timed discrete logarithms* [9] (VTD). Specifically, Alice and Bob first exchange their VTDs for their refund transactions, with timeouts T_0 and T_1, respectively. Alice then generates a statement-witness pair (Y, y), and both parties exchange pre-signatures on their swap transactions using the same statement Y. Once Alice discloses the witness y, both parties can redeem their intended assets; otherwise, each reclaims their original assets by opening their VTD after the respective timeouts. However, the universal atomic swap remains vulnerable to *griefing attacks*.

Griefing attacks (also called *sore-loser attacks*) arise when a deviating party unilaterally aborts the protocol, leading to the temporary lockup of other parties' assets until the timeout expires. For example, Alice may refuse to redeem Bob's asset if its market price declines, or Bob may fail to lock his asset if Alice's asset depreciates. Although atomicity prevents direct asset loss, the locked assets may incur indirect economic costs during a waiting period (e.g., 12 hours), such as foregone interest or additional transaction fees. To mitigate the attacks, several works [10], [11] proposed hedged swap protocols using premium-based mechanisms, where a deviating party's premium is transferred to the compliant party as compensation upon abortion. Further studies introduced griefing-free constructions [12], [13], fast settlement schemes [14], [15], and dynamic premium adjustments [16] to optimize the efficiency and fairness of premium-based mechanisms. However, these solutions rely on the scripting capabilities of blockchains and are thus incompatible with scriptless blockchains. Alternative approaches [17], [18] rely on a third-party intermediary, which is presumed to remain honest and not abort the swap due to its economic and reputation incentives, but this reintroduces centralization risks such as single points of failure.

In this paper, we bridge this gap by proposing the first universal hedged swap protocol that mitigates griefing attacks without relying on scripting capabilities or third-party inter-

mediaries.

A. Key Challenges

We identify two challenges arising from the removal of on-chain scripts, called the *timeout race attacks* and the *timeout overlap dilemma*.

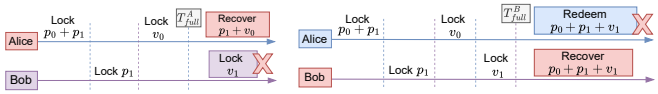
Timeout race attacks. In HTLC-based protocols, timelock scripts impose strict expiration constraints to guarantee asset *safety*: if assets are not redeemed before the pre-determined timeout, the locked assets can *only* be refunded to their original owners, ensuring no compliant party loses assets. In contrast, universal swap protocols replace on-chain timelock scripts with timed signatures, which enforce a lower bound on the opening time of refund signatures by the *privacy* property of VTD. However, such protocols lack an explicit upper bound on swap transaction validity periods, potentially allowing a deviating party to redeem the counterparty's asset even after the designated timeout.

The absence of upper bound constraints opens the door to a novel vulnerability, which we term *timeout race attacks*: a deviating party can trigger a race condition by posting a pre-timeout transaction after the designated timeout, conflicting with a post-timeout transaction; if the pre-timeout transaction is confirmed, atomicity is compromised. For instance, consider a swap scenario with Alice's timeout $T_0 = T_1 + \delta + \varphi$ and Bob's timeout T_1, where δ and φ are the upper bound of network delivery delay and confirmation latency, respectively, in the underlying blockchain. Alice can delay redeeming Bob's coins v_1 after T_1, and subsequently reclaim her coins v_0 after T_0, ultimately acquiring both v_0 and v_1.

PipeSwap [19] introduces a two-hop completion mechanism to mitigate the timeout race attacks[1]. It leverages the multi-input multi-output (MIMO) capability of UTXO-based blockchains to split Bob's coins v_1 into two outputs, ϵ and $v_1 - \epsilon$, where ϵ is designated to predetermine the final direction of the coins v_1 flow. Alice is required to post a pre-swap transaction that spends the ϵ output and reveals her witness at least $3\delta + 3\varphi$ before T_1, thereby enabling Bob to redeem Alice's coins v_0 using the witness before T_1. Otherwise (i.e., if the pre-swap transaction is not confirmed), Bob can post a pre-refund transaction time $2\delta + 2\varphi$ before T_1 to transfer the ϵ output to a new address, which prevents Alice from redeeming Bob's v_1 after T_1. However, this solution applies only to basic atomic swaps with a single asset and a single timeout for each party. It does not extend to hedged swap protocols that involve multiple assets (i.e., premium and principal) with overlapping refund timeouts.

In fact, we observe a straightforward yet effective countermeasure for mitigating the timeout race attacks in basic atomic swaps: extending the timeout interval to at least $2\delta + \varphi$, i.e., $T_0 - T_1 \geq 2\delta + \varphi$. This method ensures that Bob always has sufficient time (i.e., at least $\geq \delta + \varphi$) to redeem Alice's coins, even when Alice attempts to delay. We use a game-theoretic analysis to prove that, under this timing constraint, no party can enhance their payoff by deviating from the protocol (see Appendix E). We do not claim any novelty for this interval setting because we consider it straightforward.

[1]Referred to as *double-claiming attacks* in PipeSwap [19].



(a) Alice locks v_0 until before T^A_{full}, then reclaims $p_1 + v_0$, causing Bob to lose p_1.

(b) Bob locks v_1 until just before T^B_{full}, then reclaims $p_0 + p_1 + v_1$, resulting in Alice losing p_0.

Fig. 1: Timeout race vulnerability when $T_{prem} > T_{full}$.

(a) Alice fails to redeem Bob's v_1 but reclaims her $p_0 + p_1$.

(b) Bob fails to lock his v_1 but reclaims his p_1.

Fig. 2: Premium refund vulnerability when $T_{prem} < T_{full}$.

Nevertheless, we note that although this timing interval setting mitigates the timeout race attacks in single-timeout protocols, there exists a scheduling dilemma when the swap protocols have multiple overlapping refund timeouts.

Timeout overlap dilemma. Hedged swap protocols [10], [11] mitigate griefing attacks by requiring parties to lock premiums before locking their principals. To ensure the safety of both premiums and principals, two separate refund timeouts are necessary:

1) A *premium timeout* T_{prem} allows a compliant party to reclaim its premium if the counterparty fails to lock its principal;
2) A *principal-plus-premium timeout* T_{full} allows a compliant party to reclaim its principal and obtain the counterparty's premium if the principal is not redeemed.

While contract-based swap protocols enforce precise upper and lower bounds on transaction timeouts to guarantee the correct temporal ordering of the swap process, universal atomic swap protocols lack the ability to impose upper bounds on transaction confirmation latency and validity windows, thereby creating a *scheduling dilemma*:

- If $T_{prem} > T_{full}$, a deviating party can exploit the *timeout race vulnerability* to delay locking its principal until T_{full}, and then immediately reclaim both its principal and the counterparty's premium, potentially causing the compliant party to lose their premium. For instance, Figure 1a shows that Alice delays locking her v_0 until T^A_{full} and then reclaims both v_0 and Bob's premium p_1, which results in Bob failing to lock v_1 and thereby losing p_1. The core reason is the absence of an upper bound on the confirmation latency of Alice's principal-lock transaction. Similarly, Figure 1b shows a symmetric scenario where Bob delays locking his v_1 until T^B_{full} and then reclaims v_1 and Alice's premium $p_0 + p_1$, leading to Alice failing to redeem v_1 and thereby losing p_0.
- If $T_{prem} < T_{full}$, a deviating party can exploit a *premium refund vulnerability* to prematurely reclaim its premium,

thereby allowing it to unilaterally abort the swap without penalty. For instance, Figure 2a shows that Alice successfully reclaims her premium without redeeming Bob's v_1, due to the absence of an upper bound on the validity window of Alice's premium-refund transaction. Figure 2b also shows that Bob reclaims his premium without locking his v_1.

To address this problem, an intuition insight is to eliminate the *premium timeout* by coupling a party's principal and the counterparty's premium into a single lock transaction. However, we observe the removal of premium timeout compromises premium safety; for example, Alice may successfully lock Bob's premium, but then prematurely withdraw her premium, causing Bob's principal lock to fail and consequently resulting in the loss of his premium.

Our solution therefore requires Alice to first lock her premium in a shared address controlled by both parties. We utilize a hard relation generated by Alice for her premium refund instead of premium timeout setting. Specifically, Alice locks her premium using a statement and can later refund it using the corresponding witness. Consequently, once Alice has locked Bob's premium, Bob can also lock Alice's premium from the shared address. If malicious Alice refunds her premium using her witness after locking Bob's premium, Bob can redeem his premium and obtain Alice's principal using the same witness.

High-Value Asset Swaps. In high-value asset swaps (e.g., high-notional DeFi trades), there may be no overlap between the smallest amount one party is willing to accept as a premium and the largest amount the other party is willing to expose to lock-up risk. For example, suppose Alice wishes to swap asset V_0 for Bob's asset V_1. Bob may require a premium of at least 10 units of V_1 to compensate for the risk that Alice aborts, while Alice may only be willing to expose at most 1 unit of V_1, since a larger premium would still leave her vulnerable to griefing if Bob aborts. We therefore propose a *round-based HedgeSwap* that enables both parties to bootstrap the swap with a small, mutually acceptable premium and gradually increase it over multiple rounds until reaching the desired premium amount.

However, the simple coupling approach does not generalize to round-based HedgeSwap, which involves multiple, sequenced premium-locking rounds, since the principal and the accumulated premium should not be coupled into a single transaction and locked. Therefore, to resolve the dilemma of overlapping refund timeouts, where multiple refund transactions may simultaneously remain valid for the same assets, we propose a *premium migration mechanism*. The core idea is to merge newly locked premium with previously locked premium as inputs into a single transaction so that once a new refund transaction is deemed valid, any previously valid refund transaction becomes invalid. Moreover, we set a minimal timeout interval of $2\delta + 2\varphi$ for each round, ensuring that even if a deviating party delays posting its premium lock transaction in any round, the compliant party still has at least $\delta + \varphi$ time to proceed to the next round. This mechanism can be applied to any universal protocol that faces the multiple-timeout dilemma.

B. Our Contribution

Our main contributions are illustrated as follows.

- We propose *HedgeSwap*, the first universal hedged atomic swap protocol that mitigates griefing attacks and requires only basic signature verification from the underlying blockchain, which ensures a compliant party can receive compensation if its locked asset is not redeemed. We further develop *round-based HedgeSwap* to support high-value asset swaps where one party's smallest acceptable premium does not overlap with the other party's maximum lockable premium. We model these protocols within the universal composability framework [20], [21] and provide formal security proofs.
- We identify two key challenges in universal hedged swap protocols: (i) the *timeout race attack*, which arises due to the absence of an upper bound on the transaction validity period; and (ii) the *timeout overlap dilemma*, which arises from multiple overlapping refund windows. To address these challenges, we eliminate the premium timeout and set a minimum timeout interval of $2\delta + \varphi$ in HedgeSwap. We further introduce a premium migration mechanism and set a timeout interval of $2\delta + 2\varphi$ for each round in round-based HedgeSwap.
- We implement our HedgeSwap and round-based HedgeSwap protocols, and conduct extensive experiments to evaluate their performance in terms of off-chain and on-chain costs. The results demonstrate that our protocols exhibit high efficiency and practical applicability. Specifically, HedgeSwap achieves the running time of less than 0.5 seconds and communication costs of less than 600KB, while reducing gas costs by $2.69\times$ compared with existing contract-based solutions. Round-based HedgeSwap achieves a running time of less than 1.3 seconds and communication of less than 1,800KB, and incurs about $4.77\times$ the gas cost of HedgeSwap in a five-round setting.

II. Preliminary

We now describe the cryptographic primitives used in our work. Formal descriptions can be found in Appendix A.

Hard Relations. A hard relation $\mathcal{R}$ is defined by a statement-witness pair $(Y, y) \in \mathcal{R}$. We denote $L_{\mathcal{R}}$ as the language associated with $\mathcal{R}$, defined by $L_{\mathcal{R}} := \{Y \mid \exists y \text{ s.t. } (Y, y) \in \mathcal{R}\}$. A relation is considered hard if it satisfies: (i) There exists a PPT sampling algorithm $(Y, y) \leftarrow \mathsf{RGen}(1^\lambda)$ that outputs a statement-witness pair $(Y, y) \in \mathcal{R}$; (ii) The relation is polynomial-time decidable; (iii) For all PPT adversaries $\mathcal{A}$, the probability of $\mathcal{A}$ outputting a witness y on input Y is negligible.

Digital Signature. A signature scheme $\Sigma_{\mathsf{SIG}} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ consists of three algorithms: $(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$ generates a public-private key (pk, sk); the signing algorithm $\sigma \leftarrow \mathsf{Sign}(sk, m)$ takes sk and a message m to produce a signature σ; $\mathsf{Vrfy}(pk, m, \sigma)$

checks if σ is valid, returning 1 if valid or 0 otherwise. Σ_{SIG} ensures *correctness* and *unforgeability*.

Adaptor Signature. An adaptor signature scheme [8] integrates a signature scheme Σ_{SIG} and a hard relation $\mathcal{R}$, incorporating four algorithms $\Sigma_{\mathsf{AS}} = (\mathsf{pSign}, \mathsf{Adapt}, \mathsf{pVrfy}, \mathsf{Ext})$: $\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$ generates a pre-signature $\tilde{\sigma}$ from a private key sk, a message $m \in \{0,1\}^*$, and a statement $Y \in L_{\mathcal{R}}$. $\mathsf{pVrfy}(pk, m, Y, \tilde{\sigma})$ verifies $\tilde{\sigma}$, returning 1 if valid or 0 otherwise. $\sigma \leftarrow \mathsf{Adapt}(\tilde{\sigma}, y)$ transforms $\tilde{\sigma}$ into a full signature σ using a witness y. $y \leftarrow \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$ extracts y from σ and $\tilde{\sigma}$ such that $(Y, y) \in \mathcal{R}$. This scheme ensures the *unforgeability*, *pre-signature correctness*, *pre-signature adaptability*, and *witness extractability*.

Verifiable Timed Dlog. A verifiable timed dlog (discrete logarithm) [9] $\Sigma_{\mathsf{VTD}} = (\mathsf{Commit}, \mathsf{Verify}, \mathsf{Open}, \mathsf{ForceOp})$ involves four phases: $(C, \pi) \leftarrow \mathsf{Commit}(sk, T)$ inputs a secret value sk (generated using $\Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$) and a time parameter T, then produces a timed commitment C and a proof π. $\mathsf{Verify}(pk, C, \pi)$ checks the integrity of C and π, and accepts them by outputting 1 if sk embedded in C satisfies $pk = g^{sk}$; otherwise, it outputs 0. $(sk, r) \leftarrow \mathsf{Open}(C)$ allows the committer to reveal sk and the associated randomness r. $sk \leftarrow \mathsf{ForceOp}(C)$ extracts sk from C within the time T. A VTD scheme satisfies *soundness* and *privacy*.

Two-Party Computation. A two-party computation (2PC) protocol enables two parties to jointly compute a function over their private inputs while keeping these inputs hidden. In our setting, we use 2PC to realize three interactive protocols. The joint address creation $\Pi_{\mathsf{SIG}}^{\mathsf{JKGen}}$: on public parameters $(\mathbb{G}, g, q)$, U_0 and U_1 jointly generate a shared public key pk_{01} and private shares $sk_{01}^{(0)}$ held by U_0 and $sk_{01}^{(1)}$ held by U_1 such that $pk_{01} = g^{sk_{01}}$, where $sk_{01} = sk_{01}^{(0)} \oplus sk_{01}^{(1)}$ and $\oplus$ is $+$ for Schnorr and $\times$ for ECDSA. The joint signing $\Pi_{\mathsf{SIG}}^{\mathsf{JSign}}$: on public input a transaction tx and private inputs $sk_{01}^{(0)}, sk_{01}^{(1)}$, it outputs a joint signature σ_{tx} on tx. The joint pre-signing $\Pi_{\mathsf{AS}}^{\mathsf{JpSign}}$: on public input a transaction tx and a statement Y of a hard relation $\mathcal{R}$, and private inputs $sk_{01}^{(0)}, sk_{01}^{(1)}$, it outputs a pre-signature $\tilde{\sigma}_{tx}$ on tx. In this paper, we instantiate the 2PC protocols as those used in [19], [22].

III. Solution Overview

This section presents overviews of our HedgeSwap and round-based HedgeSwap protocols. Before illustrating the protocols, we first specify the system model, threat model, and network assumptions.

System model. We consider a scenario where Alice, denoted by U_0, holds coins v_0 on blockchain B_0, and Bob, denoted by U_1, holds coins v_1 on blockchain B_1, and they want to securely swap these coins. Alice could transfer v_0 to Bob on B_0 in the hope that Bob will transfer v_1 to Alice on B_1. If either party locks coins that are not redeemed, it should recover its assets and receive a premium as compensation.

Threat model. We consider a Byzantine adversary that can corrupt any party U_0 or U_1. A corrupted party may arbitrarily deviate from the protocol specification. For example, it may receive assets while refusing to transfer its own assets to the counterparty. We do not consider adversaries who compromise underlying blockchains themselves, such as by denial-of-service, long-range, or consensus-level attacks. We also assume miners do not censor or delay transactions.

Network assumptions. We adopt the same network assumptions as prior works [7], [19]: (i) Communication between parties is δ-synchronous, i.e., the network delivery delay is under adversarial control up to a known upper bound δ. (ii) Blockchains operate under a maximum confirmation delay φ, i.e., a valid transaction can be finally confirmed (i.e., cannot be reverted, replaced, or deleted) within time φ. If two conflicting transactions tx_1 and tx_2 with identical transaction fees become visible at times t_1 and t_2, respectively, with $t_1 < t_2$, miners always include tx_1. If they become visible simultaneously, i.e., $t_1 = t_2$, miners choose uniformly at random, so each of tx_1 and tx_2 is confirmed with probability $1/2$. (iii) All communication channels between participants are authenticated to prevent impersonation and ensure message integrity.

A. HedgeSwap Protocol

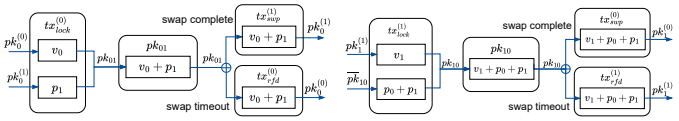
We now present an overview of our HedgeSwap. Let p denote the premium[2] and v denote the principal, with $p \ll v$. Let p_0 and p_1 denote Alice's and Bob's premium, respectively. HedgeSwap guarantees that if Alice deviates, Bob obtains p_0 as compensation, and if Bob deviates, Alice obtains p_1 as compensation. We use a superscript (j), for $j \in \{0,1\}$, to denote the address, transaction, and signature held by user U_j, and a subscript $j(1-j)$ such as pk_{01} for $j = 0$ to denote a shared address on B_j.

Similar to contract-based protocols, where all steps are hard-coded in contracts before locking assets, HedgeSwap requires that all shared addresses are created and pre-specific signatures are generated before parties lock their premiums. For clarity, we use the terms *deposit premium* and *lock principal* to distinguish two types of asset transfers, although both refer to transferring assets to a shared address.

Here, we illustrate the detailed process (see Figure 4) of our HedgeSwap as follows.

1) **Swap Setup.** Alice and Bob jointly create three shared addresses pk_{01} on B_0 and pk_{10} and $\overline{pk}_{10}$ on B_1. Alice holds shares $(sk_{01}^{(0)}, sk_{10}^{(0)}, \overline{sk}_{10}^{(0)})$ and Bob holds shares $(sk_{01}^{(1)}, sk_{10}^{(1)}, \overline{sk}_{10}^{(1)})$. Alice then generates a lock transaction $tx_{lock}^{(0)}$ (see Figure 3a), a swap transaction $tx_{swp}^{(0)}$, and a pre-refund transaction $\overline{tx}_{rfd}^{(0)}$, and she also computes a statement-witness pair $(Y, y) \in \mathcal{R}$. Bob generates a lock transaction $tx_{lock}^{(1)}$ (see Figure 3b) and a swap transaction $tx_{swp}^{(1)}$. To ensure the protocol is executed successfully, all lock transactions that use the shared address as an input are jointly signed, and all swap transactions and Alice's pre-refund transaction are also jointly pre-signed using the

[2]The premium amount can be estimated using pricing models such as the Cox-Ross-Rubinstein option model [23].



(a) Assets v_0 and p_1 flow on B_0. (b) Assets v_1 and p_0+p_1 on B_1.

Fig. 3: Inputs and outputs of transactions in HedgeSwap.

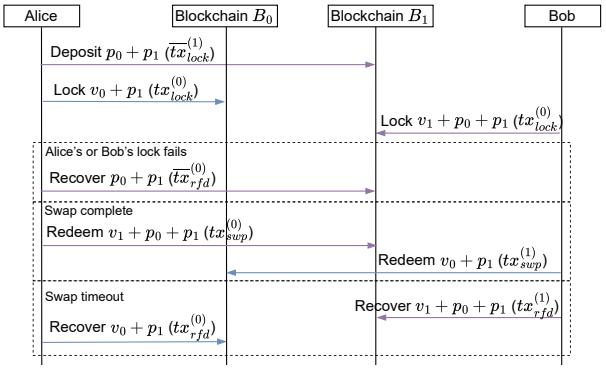
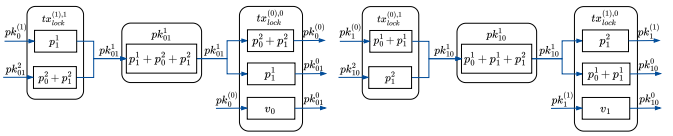


Fig. 4: An overview of HedgeSwap protocol.

statement Y. To prevent a malicious abort, Alice computes a VTD of $sk^{(0)}_{10}$ with a timeout T_1, and Bob computes a VTD of $sk^{(1)}_{01}$ with $T_0 = T_1+2\delta+\varphi$. Once all transactions and signatures are generated and verified, Alice deposits p_0+p_1 into $\overline{pk}_{10}$.

2) **Swap Lock.** In this phase, Alice and Bob lock their principal and their counterparty's premium. Alice posts $tx^{(0)}_{lock}$ on B_0 to lock her v_0 and Bob's p_1 into the address pk_{01}. Bob then posts $tx^{(1)}_{lock}$ on B_1 to lock his v_1 and Alice's p_0+p_1 to pk_{10}. If Alice is unable to lock v_0+p_1 (e.g., because Bob doesn't have enough balance p_1 from his address $pk^{(1)}_1$), Alice can post her $\overline{tx}^{(0)}_{rfd}$ using her witness y to reclaim her p_0+p_1 from $\overline{pk}_{10}$. In the case that Bob refuses to lock his v_1, Alice should wait until T_0 to post $tx^{(0)}_{rfd}$ to reclaim her principal and Bob's premium (for compensation) v_0+p_1, then using witness y to post $\overline{tx}^{(0)}_{rfd}$ to reclaim her premium p_0+p_1. Notice that Alice must only show witness y in the last step for $\overline{tx}^{(0)}_{rfd}$, otherwise Bob may extract y to post $tx^{(1)}_{swp}$ to maliciously obtain v_0+p_1 before Alice can do so at T_0.
3) **Swap Complete.** In this phase, both parties redeem their counterparty's principal and their own premium. Alice finalizes her swap transaction $tx^{(0)}_{swp}$ using her witness y and posts it on B_1 to redeem Bob's principal and her premium $v_1+p_0+p_1$. Bob then extracts y from Alice's signature and then finalizes $tx^{(1)}_{swp}$ to redeem Alice's principal and its premium v_0+p_1.
4) **Swap Timeout.** If Alice fails to complete the swap, Bob can post a refund transaction $tx^{(1)}_{rfd}$ by force-opening Alice's VTD of $sk^{(0)}_{10}$ after T_1 to reclaim his principal and Alice's premium $v_1+p_0+p_1$, where the additional p_0 (Alice's premium minus Bob's premium) compensates Bob for locking v_1. Afterwards, Alice can also post $tx^{(0)}_{rfd}$ by force-opening Bob's VTD of $sk^{(1)}_{01}$ after T_0 to reclaim v_0+p_1 locked in pk_{01}.



(a) Assets $p^2_0+p^2_1$ and p^1_1 flow on B_0. (b) Assets $p^1_0+p^1_1$ and p^2_1 flow on B_1.

Fig. 5: Inputs and outputs of lock transactions in two-round HedgeSwap.

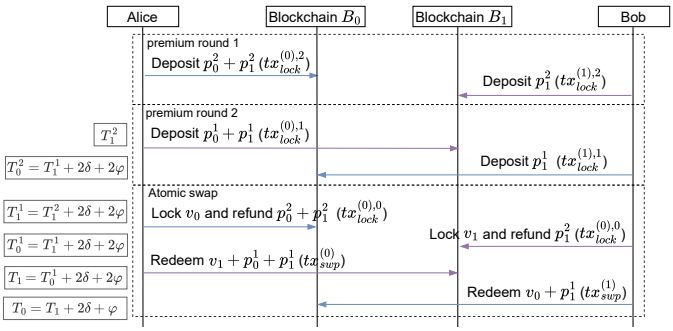


Fig. 6: An overview of two-round HedgeSwap protocol. The timeout parameters associated with lock and swap transactions are annotated in the left margin.

Security Properties. A hedged atomic swap should satisfy the following properties:

- *Atomicity.* If a compliant party transfers its asset to the counterparty, it receives the counterparty's asset; otherwise, it recovers own asset.
- *Hedge.* If a compliant party locks assets that are not redeemed, it receives sufficient compensation for the locked assets.

B. Round-based HedgeSwap Protocol

We now present an overview of our round-based HedgeSwap. Let (p^i_0, p^i_1) denote the incremental premiums associated with round index i (for $i \in \{1,\ldots,N\}$). Concretely, Alice and Bob first decide both acceptable premiums v_0/q^N[3] $= p^N_0$ and $v_1/q^N = p^N_1$, for $q>1$, where $1/q$ is the premium rate in each round. In the first round, Alice and Bob deposit their initial premiums $(v_0+v_1)/q^N$ and $(v_0)/q^N$. In the following rounds, they deposit premiums $(v_0+v_1)/q^{N-i}$ and $(v_0)/q^{N-i}$, where i is the round index starting from 0 for the first round until depositing desired premiums $(v_0+v_1)/q$ and v_0/q. After the i-th round ends, the user's premium in the $(i-1)$-th round is refunded in the $(i+1)$-th round.

For clarity, we illustrate a two-round ($N=2$) HedgeSwap protocol as follows (see Figure 6). The full round-based HedgeSwap protocol is presented in Section VI.

1) **Premium Setup and Round 1.** Alice and Bob first open six shared addresses, pk^2_{01}, pk^1_{01}, pk^0_{01} on B_0, and

[3] N can be calculated by $\log_q(\frac{v_0+v_1}{p})$, where p is the minimum acceptable premium.

pk_{10}^2, pk_{10}^1, pk_{10}^0 on B_1. Alice generates two lock transactions: $tx_{lock}^{(0),1}$ (see Figure 5b), and $tx_{lock}^{(0),0}$ (see Figure 5a), a swap transaction $tx_{swp}^{(0)}$, and a statement-witness pair $(Y, y) \in \mathcal{R}$. Bob also generates two lock transactions: $tx_{lock}^{(1),1}$ (see Figure 5a), and $tx_{lock}^{(1),0}$ (see Figure 5b), and a swap transaction $tx_{swp}^{(1)}$. Then, Alice and Bob jointly sign their lock transactions and pre-sign their swap transactions. To prevent a malicious party abort in any round, Alice generates three VTD for $sk_{10}^{(0),2}$ with a timeout T_1^1, $sk_{01}^{(0),1}$ with a timeout $T_1^2 = T_1^1 + 2\delta + \varphi$, and $sk_{10}^{(0),0}$ with a timeout $T_1 = T_1^2 + 4\delta + 4\varphi$. Similarly, Bob generates three VTD for $sk_{01}^{(1),2}$ with a timeout $T_0^1 = T_1^1 - 2\delta - 2\varphi$, $sk_{10}^{(1),1}$ with a timeout $T_0^2 = T_1^2 + 2\delta + 2\varphi$, and $sk_{01}^{(1),0}$ with $T_0 = T_1 + 2\delta + \varphi$. Once all transactions and signatures are verified, and Alice deposits $p_0^2 + p_1^2$ into pk_{01}^2 and Bob deposits p_1^2 into pk_{10}^2.

2) **Premium Round 2.** Alice posts $tx_{lock}^{(0),1}$ on B_1 to deposit her $p_0^1 + p_1^1$ together with Bob's p_1^2, previously locked at pk_{10}^2, into pk_{10}^1. Bob then posts $tx_{lock}^{(1),1}$ on B_0 to deposit his p_1^1 together with Alice's $p_0^2 + p_1^2$, previously locked at pk_{01}^2, into pk_{01}^1. Note that if Alice fails to deposit her $p_0^1 + p_1^1$, Bob can redeems his p_1^2 after T_1^2 and Alice can redeem her $p_0^2 + p_1^2$ after T_0^2. If Bob fails to deposit his p_1^1, Alice can redeem her $p_0^2 + p_1^2$ after T_0^2 and then redeem her $p_0^1 + p_1^1$ and obtain Bob's p_1^2 after T_0^1. Even if a deviating party delays posing their pre-specific lock transaction that is confirmed, the other party always has at least $\delta + \varphi$ time to post next-step lock transaction.
3) **Swap Lock.** Alice posts $tx_{lock}^{(0),0}$ on B_0 to lock her v_0 together with Bob's p_1^1, previously locked at pk_{01}^1, into the address pk_{01}^0 and refund its $p_0^2 + p_1^2$. Symmetrically, Bob posts $tx_{lock}^{(1),0}$ on B_1 to lock his v_1 together with Alice's $p_0^1 + p_1^1$ locked at pk_{10}^1 to pk_{10}^0 and refund its p_1^2.
4) Alice and Bob follow the swap complete and swap timeout phases described in Section III-A.

IV. Formal Definition

In this section, we formalize the security model and the ideal functionality of our HedgeSwap protocol. The ideal functionality of our round-based HedgeSwap protocol is deferred to Appendix B.

A. Security Model

We utilize the universal composability (UC) framework [20], specifically, the Global UC version [21], to formally model the security of our HedgeSwap protocol.

UC Security The GUC model specifies two worlds, a real world and an ideal world, denoted by $\mathsf{EXEC}_{\Pi,\mathcal{A},\mathcal{Z}}$ and $\mathsf{EXEC}_{\mathcal{F},\mathcal{S},\mathcal{Z}}$, respectively. In the real world, the parties execute the real protocol Π while interacting with an adversary $\mathcal{A}$ and an environment $\mathcal{Z}$. In the ideal world, the parties execute the ideal functionality $\mathcal{F}$, interacting with a simulator $\mathcal{S}$ and the environment $\mathcal{Z}$. The protocol Π UC-realizes the functionality $\mathcal{F}$ if for every adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that for all environments $\mathcal{Z}$, the executions $\mathsf{EXEC}_{\Pi,\mathcal{A},\mathcal{Z}}$ and $\mathsf{EXEC}_{\mathcal{F},\mathcal{S},\mathcal{Z}}$ are indistinguishable to $\mathcal{Z}$.

The blockchain functionality $\mathcal{F}_B$ interacts with users U_0 and U_1, the ideal adversary $\mathcal{S}_B$, and the environment $\mathcal{Z}$. It maintains a ledger $\mathcal{L}$ and an append-only bulletin $\mathcal{T}$. It is parameterized by a digital signature scheme $\mathsf{SIG} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$.

Interface: Initiate(pk, v), called by $\mathcal{Z}$:

1. Set the ledger $\mathcal{L} := \{(pk_1, v_1), \ldots, (pk_n, v_n)\} \in \mathbb{R}_{\geq 0}^{2n}$
2. Store and send $\mathcal{L}$ to all entities.

Interface: Publish(pk, t), called by U_0 or U_1:

1. If $\mathsf{Valid}(tx) = 1$, send $(\mathsf{publish}, tx, t)$ to $\mathcal{S}_B$.
2. Upon receiving $(\mathsf{publish}, tx, t')$ from $\mathcal{S}_B$, if $t' - t \leq \delta$, set $t := t'$; otherwise, $t := t + \delta$. For the conflicting transactions (tx_0, t_0) and (tx_1, t_1), if $t_0 < t_1$, remove (tx_1, t_1); else if $t_0 = t_1$, randomly select $b \in \{0, 1\}$ and remove (tx_b, t_b); otherwise, remove (tx_0, t_0).
3. Update $\mathcal{T} := \mathcal{T} \cup \{(tx, t)\}$ at time t.

Interface: Confirm(tx), called by U_0 or U_1 at time t''

1. If $\exists (tx, t) \in \mathcal{T}$ and $t'' - t \geq \varphi$, update $\mathcal{L}$ as $tx.pk_{in}.\mathsf{bal} := tx.pk_{in}.\mathsf{bal} - v$ and $tx.pk_{out}.\mathsf{bal} := tx.pk_{out}.\mathsf{bal} + v$
2. Otherwise, abort.

Fig. 7: Global Ideal Functionality $\mathcal{F}_B$

Adversary Model. We consider a static corruption model in which the adversary $\mathcal{A}$ can corrupt any designated users U_0 and U_1 at the beginning of the process. The corrupted party is controlled by $\mathcal{A}$ may arbitrarily deviate from the protocol (e.g., refuse to transfer assets to its counterparty).

Communication Network We adopt the same network assumptions as in prior works [7], [19]. We assume a synchronous communication network, and the execution of the protocol occurs in discrete rounds. This is modeled by the global clock functionality $\mathcal{F}_{clock}$ [24], where all honest parties are required to indicate they are ready to move to the next round before the clock can proceed. A party can abort at any round by sending a special abort message. We also assume the existence of secure message transmission channels, modeled by the ideal functionality $\mathcal{F}_{smt}$, ensuring the adversary cannot read or alter the contents of messages.

Blockchain Functionalities Consistent with prior works [7], [19], we formalize the underlying blockchain as a global functionality $\mathcal{F}_B$ with δ-synchronous communication and φ-confirmation time. An adversary $\mathcal{A}_B$ may delay or reorder transactions within δ rounds, but cannot modify or drop them, and any delivered transaction can be recorded within φ time units. $\mathcal{F}_B$ maintains a ledger $\mathcal{L}$ of balances $pk.\mathsf{bal}$ and an append-only bulletin board $\mathcal{T}$ that stores submitted transactions tx with timestamps t. $\mathcal{F}_B$ provides three interfaces: (i) $\mathsf{Valid}(tx)$ checks transaction validity (e.g., sufficient balance in $\mathcal{L}$, correct signatures, and no conflict with $\mathcal{T}$). (ii) $\mathsf{Publish}(tx, t)$ appends a valid tx to $\mathcal{T}$ at time $t' \leq t + \delta$, where the specific time t' is determined by the ideal adversary $\mathcal{S}_B$; if conflicting tx_0, tx_1 with equal transaction fees are published at the same t', $\mathcal{T}$ selects one uniformly at random. (iii) $\mathsf{Confirm}(tx)$ updates $\mathcal{L}$ by transferring coins from the input account pk_{in} to the output account pk_{out} after tx has remained in $\mathcal{T}$ for φ time. The details are provided in Figure 7.

Premium Setup Phase

- Upon receiving $(\text{pre-lock}, sid, pk_1^{(0)}, sk_1^{(0)}, p_0{+}p_1)$ from U_0 at time t:
  1. Invoke subroutine $\mathsf{Freeze}(sid, pk_1^{(0)}, \overline{pk}_{\mathcal{F}}, p_0 + p_1)$.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_1}$ at time t', where $t' \leq t + \delta + \varphi$, set $\overline{b} = 1$ and send (frz, ok) to U_0.

Swap Lock Phase

- Upon receiving $(\mathsf{lock}, sid, pk_0^{(0)}, pk_0^{(1)}, sk_0^{(0)}, sk_0^{(1)}, v_0, p_1)$ from U_0 at time t,
  1. If $\overline{b} = 1$, invoke subroutine $\mathsf{Freeze}(sid, (pk_0^{(0)}, pk_0^{(1)}), pk_{\mathcal{F}}^0, (v_0, p_1), T_0)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_0}$ at t', where $t' \leq t{+}\delta{+}\varphi$, append this entry $(sid, pk_0^{(0)}, v_0{+}p_1, pk_{\mathcal{F}}^0, T_0)$ to the list Locked, set $b_0{=}1$, and send (frz, ok) to U_0 and U_1.
  3. Upon receiving $(\mathsf{rfd}, sid, pk_0^{(0)})$ from U_0 at time t', if $t' > T_0$ and the corresponding entry is still in Locked, then invoke subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F}}^0, pk_0^{(0)}, v_0 + p_1)$, set $b_0 = 0$, and delete the entry from Locked.
- Upon receiving $(\mathsf{lock}, sid, pk_1^{(1)}, sk_1^{(1)}, v_1)$ from U_1 at time t,
  1. If $b_0 = 1$, invoke subroutine $\mathsf{Freeze}(id, (pk_1^{(0)}, \overline{pk}_{\mathcal{F}}), pk_{\mathcal{F}}^1, (v_1, p_0 + p_1), T_1)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, id)$ from $\mathcal{F}_{B_1}$ at time t', append the entry $(sid, pk_1^{(1)}, v_1 + p_0 + p_1, pk_{\mathcal{F}}^1, T_1)$ to the list Locked, set $\overline{b} = 0$ and $b_1 = 1$, and send (frz, ok) to U_0 and U_1.
  3. Upon receiving $(\mathsf{rfd}, sid, pk_1^{(1)})$ from U_1 at time t', if $t' > T_1$ and the corresponding entry is still in Locked, then invoke subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F}}^1, pk_1^{(1)}, v_1 + p_0 + p_1)$, set $b_1 = 0$, and delete the entry from Locked.
- Upon receiving $(\text{pre-rfd}, sid, pk_1^{(0)})$ from U_0 at time t, refund
  1. If $\overline{b} = 1$, invoke subroutine $\mathsf{Unfreeze}(sid, \overline{pk}_{\mathcal{F}}, pk_1^{(0)}, p_0 + p_1)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_1}$, set $\overline{b} = 0$ and send $(\text{pre-rfd}, ok)$ to U_0 and U_1.

Swap Complete

- Upon receiving $(\mathsf{swp}, sid, pk_1^{(0)})$ from U_0 at time t,
  1. If $t < T_1 \wedge b_1 = 1$, invoke subroutine $\mathsf{Transfer}(id, pk_{\mathcal{F}}^1, pk_1^{(0)}, v_1 + p_0 + p_1)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_1}$, delete the entry $(sid, pk_1^{(1)}, v_1 + p_0 + p_1, pk_{\mathcal{F}}^1, T_1)$, set $b_1 = 0$, and send (swp, ok) to U_0 and U_1.
- Upon receiving $(\mathsf{swp}, sid, pk_0^{(1)})$ from U_1 at time t,
  1. If $b_0 = 1 \vee b_1 = 0$, invoke subroutine $\mathsf{Transfer}(id, pk_{\mathcal{F}}^0, pk_0^{(1)}, v_1 + p_1)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_0}$, delete $(sid, pk_0^{(0)}, v_0 + p_1, pk_{\mathcal{F}}^0, T_0)$, set $b_0 = 1$, and send (swp, ok) to U_0 and U_1.

Fig. 8: Ideal Functionality $\mathcal{F}_{\mathsf{HSwap}}$. The details of subroutines are provided in Figure 19 (see Appendix B).

B. Formal Functionality for HedgeSwap

We formalize the security of our HedgeSwap protocol using an ideal functionality $\mathcal{F}_{\mathsf{HSwap}}$, illustrated in Figure 8. The functionality $\mathcal{F}_{\mathsf{HSwap}}$ interacts with two users U_0 and U_1, a simulator $\mathcal{S}$, and two global blockchain functionalities $\mathcal{F}_{B_0}$ and $\mathcal{F}_{B_1}$. The functionality is parameterized by the timeouts T_0, T_1. It maintains a list Locked that stores the locked coins in the functionality, initialized to $\emptyset$. It also maintains three state variables $\overline{b}, b_0, b_1$ representing U_0's premiums, U_0's principals, and U_1's principals, respectively, and initializes them as $\perp$. $\mathcal{F}_{\mathsf{HSwap}}$ specifies four procedures as follows, each triggered by a message that includes a respective request and a session identifier sid.

Swap Setup. User U_0 initiates a premium with its pre-lock message $(\mathsf{pre\text{-}lock}, sid, pk_1^{(0)}, sk_1^{(0)}, p_0{+}p_1)$ to specify $p_0{+}p_1$ as its premium. $\mathcal{F}_{\mathsf{HSwap}}$ transfers coins $p_0 + p_1$ from account $pk_1^{(0)}$ to a specific account $\overline{pk}_{\mathcal{F}}$ controlled by $\mathcal{F}_{\mathsf{HSwap}}$.

Swap Lock. User U_0 initiates a swap amount with a lock message $(\mathsf{lock}, sid, pk_0^{(0)}, pk_0^{(1)}, sk_0^{(0)}, sk_0^{(1)}, v_0, p_1)$ to specify the principal amount v_0 in account $pk_0^{(0)}$ (owned by user U_0) and the premium amount p_1 in account $pk_0^{(1)}$ (owned by user U_1). $\mathcal{F}_{\mathsf{HSwap}}$ transfers coins v_0 and p_1 to a specific account $pk_{\mathcal{F}}^0$ controlled by $\mathcal{F}_{\mathsf{HSwap}}$ for a specified period of time T_0. After the timeout T_0, if the coins $v_1 + p_0 + p_1$ are still locked in account $pk_{\mathcal{F}}^1$, $\mathcal{F}_{\mathsf{HSwap}}$ transfers them to U_0. U_1 also sends $(\mathsf{lock}, sid, pk_1^{(0)}, pk_1^{(1)}, v_1)$ to specify the principal amount v_1 in account $pk_1^{(1)}$. $\mathcal{F}_{\mathsf{HSwap}}$ transfers coins v_1 and $p_0 + p_1$ to a specific account $pk_{\mathcal{F}}^1$ controlled by $\mathcal{F}_{\mathsf{HSwap}}$ for a time period T_1. After T_1, if the coins $v_0 + p_1$ are still locked in $pk_{\mathcal{F}}^0$, $\mathcal{F}_{\mathsf{HSwap}}$ transfers them to U_1. Alternatively, user U_0 can also sends $(\text{pre-rfd}, sid, pk_1^{(0)})$ to refund its premium.

Swap Complete. User U_0 sends the swap message $(\mathsf{swp}, sid, pk_1^{(0)})$ at time t. If $t < T_1$ and $b_1 = 1$, $\mathcal{F}_{\mathsf{HSwap}}$ transfers $v_1 + p_0 + p_1$ from $pk_{\mathcal{F}}^1$ to $pk_1^{(0)}$; otherwise, it aborts. Upon receiving the swap message $(\mathsf{swp}, sid, pk_0^{(1)})$ from U_1, $\mathcal{F}_{\mathsf{HSwap}}$ transfers coins $v_0 + p_1$ from account $pk_{\mathcal{F}}^1$ to $pk_0^{(1)}$.

V. HedgeSwap Construction

In this section, we present the HedgeSwap protocol in detail and provide a formal security analysis.

HedgeSwap enables two users, U_0 and U_1, to securely exchange assets across two blockchains B_0 and B_1 while mitigating griefing attacks. Specifically, U_0 holds principal v_0 on B_0 and wishes to swap it for U_1's principal v_1 on B_1. If U_0 (resp. U_1) locks its principal v_0 (resp. v_1) as part of the protocol, it is entitled to receive a premium p_1 (resp. p_0) as compensation.

Based on the idea mentioned in Section III-A, we now describe our HedgeSwap protocol involving four phases.

Premium Setup Phase. In this phase, U_0 transfers its premium $p_0 + p_1$ to a shared address under U_0 and U_1 joint control. To prevent any aborts maliciously, each party generates respective lock transaction and swap transaction. Specifically, Alice generates a lock transaction $tx_{lock}^{(0)}$: $(pk_0^{(0)}, pk_0^{(1)}) \xhookrightarrow{v_0, p_1} pk_{01}$, a swap transaction $tx_{swp}^{(0)}$: $pk_{10} \xhookrightarrow{v_1+p_0+p_1} pk_1^{(0)}$, and a refund transaction $\overline{tx}_{rfd}^{(0)}$: $\overline{pk}_{10} \xhookrightarrow{p_0+p_1} pk_1^{(0)}$. Bob generates a lock transaction $tx_{lock}^{(1)}$: $(pk_1^{(1)}, \overline{pk}_{10}) \xhookrightarrow{v_1, p_0+p_1} pk_{10}$, and a swap transaction $tx_{swp}^{(1)}$: $pk_{01} \xhookrightarrow{v_0+p_1} pk_0^{(1)}$. The transaction $\overline{tx}_{rfd}^{(0)}$,

Global Input: $(\mathbb{G}, g, q)$, amounts (v_0, v_1, p_0, p_1), public keys: $(pk_0^{(0)}, pk_0^{(1)}, pk_1^{(0)}, pk_1^{(1)})$, time parameters: (T_0, T_1) with $T_0 = T_1 + 2\delta + \varphi$.
User U_0 input: $sk_0^{(0)}$ and $sk_1^{(0)}$, User U_1 input: $sk_0^{(1)}$ and $sk_1^{(1)}$

Premium Setup Phase

User U_0 and U_1 generate their pre-specific transactions, and then U_0 deposits its premium.

1. User U_0 and U_1 execute the $\Pi_{\mathsf{SIG}}^{\mathsf{JKGen}}$ joint address generation protocol with the common input $(\mathbb{G}, g, q)$, and output $\big(pk_{01}^{(0)}, sk_{01}^{(0)}\big)$, $\big(pk_{10}^{(0)}, sk_{10}^{(0)}\big)$, and $\big(\overline{pk}_{10}^{(0)}, \overline{sk}_{10}^{(0)}\big)$ to user U_0, $\big(pk_{01}^{(1)}, sk_{01}^{(1)}\big)$, $\big(pk_{10}^{(1)}, sk_{10}^{(1)}\big)$, and $\big(\overline{pk}_{10}^{(1)}, \overline{sk}_{10}^{(1)}\big)$ to user U_1.
2. Users U_0 and U_1 generates their pre-specific transactions:
   - User U_0 generates a pre-refund transaction $\overline{tx}_{rfd}^{(0)} := \big(\overline{pk}_{10}, pk_1^{(0)}, p_0{+}p_1\big)$, a lock transaction $tx_{lock}^{(0)} := \big(\big(pk_0^{(0)}, pk_0^{(1)}\big), pk_{01}, \big(v_0, p_1\big)\big)$, and a swap transaction $tx_{swp}^{(0)} := \big(pk_{10}, pk_1^{(0)}, v_1{+}p_0{+}p_1\big)$. User U_0 also computes a timed commitment $\big(C^{(1)}, \pi^{(1)}\big) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\big(sk_{10}^{(0)}, T_1\big)$ and a statement and witness pair $(Y, y) \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{RGen}(1^\lambda)$. User U_0 then sends the transactions $\big(\overline{tx}_{rfd}^{(0)}, tx_{lock}^{(0)}, tx_{swp}^{(0)}\big)$, the commitment $\big(C^{(1)}, \pi^{(1)}\big)$, and the statement Y to U_1;
   - User U_1 generates a lock transaction $tx_{lock}^{(1)} := \big(\big(pk_1^{(1)}, \overline{pk}_{10}\big), pk_{10}, \big(v_1, p_0{+}p_1\big)\big)$ and a swap transaction $tx_{swp}^{(1)} := \big(pk_{01}, pk_0^{(1)}, v_0{+}p_1\big)$. U_1 also computes a time commitment $\big(C^{(0)}, \pi^{(0)}\big) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\big(sk_{01}^{(1)}, T_0\big)$, and then sends $\big(tx_{lock}^{(1)}, tx_{swp}^{(1)}\big)$ and $\big(C^{(0)}, \pi^{(0)}\big)$ to U_0.
3. User U_0 and U_1 checks the received transactions and commitment:
   - U_0 checks if transactions $\big(tx_{lock}^{(1)}, tx_{swp}^{(1)}\big)$ are well formed (i.e., satisfying premium mechanism) and $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\big(C^{(0)}, \pi^{(0)}, pk_{01}^{(1)}\big) = 1$, and then starts solving $\Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(0)}\big)$; otherwise, aborts.
   - U_1 checks if transactions $\big(\overline{tx}_{rfd}^{(0)}, tx_{lock}^{(0)}, tx_{swp}^{(0)}\big)$ are well formed and $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\big(C^{(1)}, \pi^{(1)}, pk_{10}^{(0)}\big) = 1$, and then start solving $\Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(1)}\big)$, compute a signature $\sigma_{lock}^{(0),1} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_0^{(1)}, tx_{lock}^{(0),1}\big)$, and send $\sigma_{lock}^{(0),1}$ to U_0; otherwise, aborts.
4. Users U_0 and U_1 execute the joint signing protocol $\Pi_{\mathsf{SIG}}^{\mathsf{JSign}}$ with input $\big(\overline{sk}_{01}^{(0)}, \overline{sk}_{01}^{(1)}, tx_{lock}^{(1)}\big)$ and output a signature $\overline{\sigma}_{lock}^{(1)}$, and execute the joint pre-signing protocol $\Pi_{\mathsf{AS}}^{\mathsf{JpSign}}$ with input $\big(\overline{sk}_{10}^{(0)}, \overline{sk}_{10}^{(1)}, Y, \overline{tx}_{rfd}^{(0)}\big)$, $\big(sk_{01}^{(0)}, sk_{01}^{(1)}, Y, tx_{swp}^{(0)}\big)$, and $\big(sk_{10}^{(0)}, sk_{10}^{(1)}, Y, tx_{swp}^{(1)}\big)$, and output pre-signatures $\tilde{\overline{\sigma}}_{rfd}^{(0)}$, $\tilde{\sigma}_{swp}^{(0)}$, and $\tilde{\sigma}_{swp}^{(1)}$, respectively.
5. User U_0 generates a premium lock transaction $\overline{tx}_{lock}^{(0)} := \big(pk_1^{(0)}, \overline{pk}_{10}, p_0{+}p_1\big)$ and computes $\overline{\sigma}_{lock}^{(0)} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_1^{(0)}, \overline{tx}_{lock}^{(0)}\big)$. It posts $\big(\overline{tx}_{lock}^{(0)}, \overline{\sigma}_{lock}^{(0)}\big)$ on B_1.

Swap Lock Phase

User U_0 locks its principal v_0 and user U_1 locks its principal v_1.

1. User U_0 computes $\sigma_{lock}^{(0),0} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_0^{(0)}, tx_{lock}^{(0)}\big)$. U_0 posts $\big(tx_{lock}^{(0)}, \big(\sigma_{lock}^{(0),0}, \sigma_{lock}^{(0),1}\big)\big)$ on B_0. If $tx_{lock}^{(0)}$ is not confirmed before $T_1 - 2\delta - 2\varphi$ (e.g., because U_1 deviates and has insufficient balance at $pk_0^{(1)}$), U_0 computes $\overline{\sigma}_{rfd}^{(0)} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\overline{\sigma}}_{rfd}^{(0)}, y\big)$ and posts $\big(\overline{tx}_{rfd}^{(0)}, \overline{\sigma}_{rfd}^{(0)}\big)$ on B_1.
2. Users U_1 computes $\sigma_{lock}^{(1)} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_1^{(1)}, tx_{lock}^{(1)}\big)$. U_1 posts $\big(tx_{lock}^{(1)}, \big(\sigma_{lock}^{(1)}, \overline{\sigma}_{lock}^{(1)}\big)\big)$ on B_1. If $tx_{lock}^{(1)}$ is not confirmed (e.g., because U_0 deviates and has already posted $\big(\overline{tx}_{rfd}^{(0)}, \overline{\sigma}_{rfd}^{(0)}\big)$), then U_1 computes $y \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Ext}\big(\overline{\sigma}_{rfd}^{(0)}, \tilde{\overline{\sigma}}_{rfd}^{(0)}, Y\big)$, and $\sigma_{swp}^{(1)} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{(1)}, y\big)$ and posts $\big(tx_{swp}^{(1)}, \sigma_{swp}^{(1)}\big)$ on B_0.

Swap Complete Phase

User U_0 redeems U_1's v_1 and reclaim its $p_0 + p_1$, and then user U_1 redeems U_0's v_0 and reclaim its p_1.

1. User U_0 computes $\sigma_{swp}^{(0)} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}(\tilde{\sigma}_{swp}^{(0)}, y)$ and posts $(tx_{swp}^{(0)}, \sigma_{swp}^{(0)})$ on B_1 before $T_1 - \delta - \varphi$.
2. User U_1 computes $y \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Ext}\big(\sigma_{swp}^{(0)}, \tilde{\sigma}_{swp}^{(0)}, Y\big)$, and computes $\sigma_{swp}^{(1)} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{(1)}, y\big)$ and posts $\big(tx_{swp}^{(1)}, \sigma_{swp}^{(1)}\big)$ on B_0 before $T_0 - \delta - \varphi$.

Swap Timeout Phase

User U_1 reclaims its principal v_1 and obtains U_0's premium $p_0 + p_1$, while user U_0 reclaims its v_0 and U_1's p_1.

- At time T_1, U_1 generates $tx_{rfd}^{(1)} := \big(pk_{10}, pk_1^{(1)}, v_1{+}p_0{+}p_1\big)$, and computes $sk_{10}^{(0)} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(1)}\big)$ and $sk_{10} := sk_{10}^{(0)} \oplus sk_{10}^{(1)}$, $\sigma_{rfd}^{(1)} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{10}, tx_{rfd}^{(1)}\big)$. It then posts $\big(tx_{rfd}^{(1)}, \sigma_{rfd}^{(1)}\big)$ on B_1.
- At time T_0, U_0 generates $tx_{rfd}^{(0)} := \big(pk_{01}, pk_0^{(0)}, v_0{+}p_1\big)$, and computes $sk_{01}^{(1)} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(0)}\big)$ and $sk_{01} := sk_{01}^{(0)} \oplus sk_{01}^{(1)}$, $\sigma_{rfd}^{(0)} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{01}, tx_{rfd}^{(0)}\big)$. It posts $\big(tx_{rfd}^{(0)}, \sigma_{rfd}^{(0)}\big)$ on B_0.

Fig. 9: HedgeSwap Protocol. The subroutines of 2PC protocols are provided in Figure 10.

// The 2PC protocol $\Pi_{\mathsf{SIG}}^{\mathsf{JKGen}}$

1. Party U_0 computes $(pk_{01}^0, sk_{01}^0) \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$ and sends pk_{01}^0 to U_1.
2. Party U_1 computes $(pk_{01}^1, sk_{01}^1) \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$ and sends pk_{01}^1 to U_0.
3. Parties U_0 and U_1 generates the shared address pk_{01}.

// The 2PC protocol $\Pi_{\mathsf{SIG}}^{\mathsf{JSign}}$

It takes private inputs sk_{01}^0 and sk_{01}^1 held by parties U_0 and U_1 respectively, and public input tx:

1. Set secret key as $sk_{01} := sk_{01}^0 \oplus sk_{01}^1$
2. Compute signature $\sigma_{tx} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}(sk_{01}, tx)$ and sends σ_{tx} to both parties U_0 and U_1.
3. Parties U_0 and U_1 respectively check if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{01}, tx, \sigma_{tx}) = 1$, and stop otherwise.

// The 2PC protocol $\Pi_{\mathsf{AS}}^{\mathsf{JpSign}}$

It takes private inputs sk_{01}^0 and sk_{01}^1 held by parties U_0 and U_1 respectively, and public inputs (tx, Y):

1. Set secret key as $sk_{vu} := sk_{01}^0 \oplus sk_{01}^1$
2. Compute signature $\tilde{\sigma}_{tx} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{pSign}(sk_{01}, tx, Y)$ and sends $\tilde{\sigma}_{tx}$ to both parties U_0 and U_1.
3. Parties U_0 and U_1 respectively check if $\Sigma_{\mathsf{AS}}.\mathsf{pVrfy}(pk_{01}, tx, \sigma_{tx}, Y) = 1$, and stop otherwise.

Fig. 10: Subroutines of 2PC Protocols

$tx_{swp}^{(0)}$, and $tx_{swp}^{(1)}$ are jointly pre-signed with secret keys $\overline{sk}_{10}$, sk_{10}, sk_{01}, respectively, using a hard relation $(Y, y) \in \mathcal{R}$ generated by U_0. To ensure that the frozen coins are refunded as expected, U_0 generates a VTD embedding $sk_{10}^{(0)}$, which allows U_1 obtains $v_1 + p_0 + p_1$ after a timeout T_1, and U_1 generates a VTD embedding $sk_{01}^{(1)}$, which allows U_0 obtains $v_0 + p_1$ after a timeout $T_0 = T_1 + 2\delta + \varphi$. Finally, U_0 deposits $p_0 + p_1$ into $\overline{pk}_{10}$ on B_1.

Swap Lock Phase. In this phase, U_0 and U_1 lock their principal with counterparty's premium using lock transactions $tx_{lock}^{(0)}$ and $tx_{lock}^{(1)}$, respectively. If U_1 doesn't have enough balance p_1 to be locked, U_0 can reclaim its premium $p_0 + p_1$ locked in $\overline{pk}_{10}$ by signing the refund transaction $\overline{tx}_{rfd}^{(0)}$ using its witness y. After U_0 properly locks U_1's premium p_1, U_1 posts $tx_{lock}^{(1)}$ on B_1 to lock its v_1 and U_0's $p_0 + p_1$ to pk_{10}.

Swap Complete Phase. U_0 finalizes a valid swap transaction $tx_{swp}^{(0)}$ by using its witness y to redeem U_1's v_1 and its $p_0 + p_1$ before $T_1 - \delta - \varphi$. User U_1 then extracts y and uses it to compute a valid $tx_{swp}^{(1)}$, and post it to redeem U_0's v_0 and its p_1.

Swap Timeout. If U_0 fails to complete the swap, U_1 can reclaim its principal and U_0's premium $v_1 + p_0 + p_1$ by force-opening U_0's VTD to post the refund transaction $tx_{rfd}^{(1)}$ after T_1. Afterwards, U_0 can also reclaim $v_0 + p_1$ by force-opening U_1's VTD to post the refund transaction $tx_{rfd}^{(0)}$ after T_0.

Remark 1. *The time parameters are defined as follows. Let t be the time at which the swap starts. It should hold $T_1 > t + 4\delta + 4\varphi$ so that the time interval is sufficiently large to confirm the lock transactions $tx_{lock}^{(0)}$ and $tx_{lock}^{(1)}$, as well as the swap transactions $tx_{swp}^{(0)}$ and $tx_{swp}^{(1)}$ on the blockchains. Additionally, we require that $T_0 \geq T_1 + 2\delta + \varphi$ against the timeout race attacks. Furthermore, to prevent U_0 from maliciously locking U_1's premium p_1, user U_1 should only transfer its premium p_1 to the address $pk_1^{(1)}$ after observing that U_0 has posted a pre-refund transaction on B_1.*

A formal proof of the following theorem is provided in Appendix C.

Theorem 1. *If Σ_{AS} is a secure adaptor signature scheme w.r.t. a secure digital signature scheme Σ_{SIG} and a hard relation $\mathcal{R}$, Σ_{VTD} is a secure verifiable timed dlog scheme, Π_{SIG} and Π_{AS} are UC-secure 2PC protocols, respectively. Then our HedgeSwap, UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{RHSwap}}$ with access to the functionalities ($\mathcal{F}_{clock}$, $\mathcal{F}_{smt}$, $\mathcal{F}_B$).*

VI. Round-based HedgeSwap Construction

In this section, we present the round-based HedgeSwap protocol in detail and provide a formal security analysis.

Round-based HedgeSwap allows two users, U_0 and U_1, to deposit incremental premiums over multiple rounds until reaching the desired premium amounts. Based on the idea mentioned in Section III-B, U_0 and U_1 deposit their initial premiums $p_0^N + p_1^N$ and p_1^N until the N-th round to lock $p_0^1 + p_1^1 = (v_0 + v_1)/q$ and $p_1^1 = v_1/q$. Note that, in each round, users lock their premiums on alternating blockchains (e.g., U_0 locks $p_0^N + p_1^N$ on B_0 in the first round and locks $p_0^{N-1} + p_1^{N-1}$ on B_1 in the next round). For clarity, we index the N rounds by $i \in \{0, 1, \ldots, N-1\}$, where $i = 0$ denotes the first round and $i = N-1$ denotes the final round. We use the index $i = N$ to denote the basic swap lock phase. For convenience, we use the first superscript (0) and (1) to indicate the element held by U_0 and U_1, respectively, and the second superscript $N-i \in \{0, 1, \ldots, N\}$ to denote a reverse phase index, where $N-i = N$ corresponds to the first round, $N-i = 1$ corresponds to the final round, and $N-i = 0$ denotes the basic swap lock phase. We denote the set of odd and even rounds as $\mathcal{R}_{\text{odd}} = \{i \mid N-i \text{ is odd}\}$ and $\mathcal{R}_{\text{even}} = \{i \mid N-i \text{ is even}\}$. Let $j \in \{0, 1\}$ be defined by $j = 0$ if $i \in \mathcal{R}_{\mathsf{even}}$ and $j = 1$ otherwise.

We now illustrate the round-based HedgeSwap protocol as follows.

Global Input: $(\mathbb{G}, g, q)$, amounts (v_0, v_1, p_0^i, p_1^i) for $i \in \{1, \ldots, N\}$, public keys: $(pk_0^{(0)}, pk_0^{(1)}, pk_1^{(0)}, pk_1^{(1)})$, time parameters: (T_0^i, T_1^i) for $i \in \{0, \ldots, N\}$.

User U_0 input: $sk_0^{(0)}$ and $sk_1^{(0)}$, User U_1 input: $sk_0^{(1)}$ and $sk_1^{(1)}$

Premium Setup Phase

1. Users U_0 and U_1 jointly execute the protocol $\Pi_{\mathsf{SIG}}^{\mathsf{JKGen}}$ with common input. For each $i \in \{0, \ldots, N\}$, the protocol outputs $(pk_{01}^{(0),i}, sk_{01}^{(0),i})$ and $(pk_{10}^{(0),i}, sk_{10}^{(0),i})$ to U_0, $(pk_{01}^{(1),i}, sk_{01}^{(1),i})$ and $(pk_{10}^{(1),i}, sk_{10}^{(1),i})$ to U_1.
2. Users U_0 and U_1 generates their pre-specific transactions and timed commitments.
   - U_0 generates $N-1$ premium lock transactions as follows:
     - For $i \in [2, N-1]$, $tx_{lock}^{(0),N-i} := \big(\big(pk_j^{(0)}, pk_{j(1-j)}^{N-i+1}\big),\ \big(pk_{j(1-j)}^{N-i}, pk_j^{(0)}\big),\ \big(p_0^{N-i}+p_1^{N-i}+p_1^{N-i+1},\ p_0^{N-i+2}+p_1^{N-i+2}\big)\big)$.
     - For $i = 1$, $tx_{lock}^{(0),N-i} := \big(\big(pk_j^{(0)}, pk_{j(1-j)}^{N-i+1}\big),\ pk_{j(1-j)}^{N-i},\ \big(p_0^{N-i} + p_1^{N-i} + p_1^{N-i+1}\big)\big)$.
   - U_0 also generates a principal lock transaction $tx_{lock}^{(0),0} := \big(\big(pk_0^{(0)}, pk_{01}^1\big),\ \big(pk_{01}^0, pk_0^{(0)}\big),\ \big(v_0 + p_1^1, p_0^2 + p_1^2\big)\big)$ and a swap transaction $tx_{swp}^{(0)} := \big(pk_{10}^0,\ pk_1^{(0)},\ v_1 + p_0^1 + p_1^1\big)$.
   - For each $i \in [0, N]$, U_0 computes $\big(C^{(1),i}, \pi^{(1),i}\big) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\big(sk_{(1-j)j}^{(0),i}, T_1^i\big)$.
   - U_0 generates a statement and witness pair $(Y, y) \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{RGen}(1^\lambda)$.
   - U_0 then sends $tx_{lock}^{(0),i}$ for $i \in [0, N-1]$, $tx_{swp}^{(0)}$ and $\big(C^{(1),i}, \pi^{(1),i}\big)$ for $i \in [0, N]$ to U_1.
   - Similarly, U_1 generates $N-1$ premium lock transactions as follows:
     - For $i \in [2, N-1]$, $tx_{lock}^{(1),N-i} := \big(\big(pk_{1-j}^{(1)}, pk_{(1-j)j}^{N-i+1}\big),\ \big(pk_{(1-j)j}^{N-i}, pk_{1-j}^{(1)}\big),\ \big(p_1^{N-i}+p_0^{N-i+1}+p_1^{N-i+1}, p_1^{N-i+2}\big)\big)$.
     - For $i = 1$, $tx_{lock}^{(1),N-i} := \big(\big(pk_{1-j}^{(1)}, pk_{(1-j)j}^{N-i+1}\big),\ pk_{(1-j)j}^{N-i},\ p_1^{N-i}+p_0^{N-i+1}+p_1^{N-i+1}\big)$.
   - U_0 also generates a principal lock transaction $tx_{lock}^{(1),0} := \big(\big(pk_1^{(1)}, pk_{10}^1\big),\ \big(pk_{10}^0, pk_1^{(1)}\big),\ \big(v_1 + p_0^1 + p_1^1, p_1^2\big)\big)$ and a swap transaction $tx_{swp}^{(1)} := \big(pk_{01}^0,\ pk_0^{(1)},\ v_0 + p_1^1\big)$,
   - For each $i \in [0, N]$, U_1 computes $\big(C^{(0),i}, \pi^{(0),i}\big) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\big(sk_{j(1-j)}^{(1),i}, T_0^i\big)$.
   - U_1 then sends $tx_{lock}^{(0),i}$ for $i \in [0, N-1]$, $tx_{swp}^{(0)}$, and $\big(C^{(0),i}, \pi^{(0),i}\big)$ for $i \in [0, N]$ to U_0.
3. User U_0 and U_1 checks the received transactions and commitment:
   - U_0 checks if the received transactions $tx_{lock}^{(1),i}$ for each $i \in [0, N-1]$ and $tx_{swp}^{(1)}$ are well formed (i.e., satisfying round-based premium mechanism) and $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\big(pk_{j(1-j)}^i, C^{(0),i}, \pi^{(0),i}\big) = 1$; otherwise, aborts.
   - U_1 checks if transactions $tx_{lock}^{(0),i}$ for each $i \in [0, N-1]$ and $tx_{swp}^{(0)}$ are well formed, and $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\big(pk_{(1-j)j}^i, C^{(1),i}, \pi^{(1),i}\big) = 1$; otherwise, aborts.
4. For each $i \in [1, N]$, users U_0 and U_1 run $\Pi_{\mathsf{SIG}}^{\mathsf{JSign}}$ with input $\big(sk_{j(1-j)}^{(0),i}, sk_{j(1-j)}^{(1),i}, tx_{lock}^{(0),i}\big)$ and $\big(sk_{(1-j)j}^{(0),i}, sk_{(1-j)j}^{(1),i}, tx_{lock}^{(1),i}\big)$, and obtains a signature $\sigma_{lock}^{(0),i'}$ and $\sigma_{lock}^{(1),i'}$.
5. U_0 and U_1 then run $\Pi_{\mathsf{AS}}^{\mathsf{JpSign}}$ with input $\big(sk_{10}^{(0)}, sk_{10}^{(1)}, Y, tx_{swp}^{(0)}\big)$, and $\big(sk_{01}^{(0)}, sk_{01}^{(1)}, Y, tx_{swp}^{(1)}\big)$, and obtain pre-signatures $\tilde{\sigma}_{swp}^{(0)}$ and $\tilde{\sigma}_{swp}^{(1)}$, respectively.
6. U_0 generates a premium lock transaction $tx_{lock}^{(0),N} := \big(pk_j^{(0)}, pk_{j(1-j)}^N, p_0^N + p_1^N\big)$ and computes $\sigma_{lock}^{(0),N} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_j^{(0)}, tx_{lock}^{(0),N}\big)$. It posts $\big(tx_{lock}^{(0),N}, \sigma_{lock}^{(0),N}\big)$ on B_j.
7. U_1 generates $tx_{lock}^{(1),N} := \big(pk_{1-j}^{(1)}, pk_{(1-j)j}^N, p_1^N\big)$ and computes $\sigma_{lock}^{(1),N} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{1-j}^{(1)}, tx_{lock}^{(1),N}\big)$. It posts $\big(tx_{lock}^{(1),N}, \sigma_{lock}^{(1),N}\big)$ on B_{1-j}.

i-th Round Premium Lock Phase

For each $i \in [1, N-1]$, users U_0 and U_1 lock their $(i+1)$-th round premiums.

1. User U_0 computes $\sigma_{lock}^{(0),N-i} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_j^{(0)}, tx_{lock}^{(0),N-i}\big)$. It posts $\big(tx_{lock}^{(0),N-i}, \big(\sigma_{lock}^{(0),N-i}, \sigma_{lock}^{(0),N-i'}\big)\big)$ on B_j.
2. User U_1 computes $\sigma_{lock}^{(1),N-i} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{1-j}^{(1)}, tx_{lock}^{(1),N-i}\big)$. It posts $\big(tx_{lock}^{(1),N-i}, \big(\sigma_{lock}^{(1),N-i}, \sigma_{lock}^{(1),N-i'}\big)\big)$ on B_{1-j}.

Fig. 11: Round-based HedgeSwap (Party I)

Premium Timeout Round Index $i \in \{0, \ldots, N-1\}$ Phase

After the timeout T_1^{N-i}, if $tx_{lock}^{(0),N-i-1}$ is not confirmed, both users recover their $(i+1)$-th round premium. If $i \geq 2$, U_1 obtains U_0's p_0^{N-i+1} as compensation.

1. After the timeout T_1^{N-i}, if $i = 0$, U_1 generates $tx_{rfd}^{(1),N} := \big(pk_{(1-j)j}^{N}, pk_{1-j}^{(1)}, p_1^N\big)$; otherwise, U_1 generates $tx_{rfd}^{(1),N-i} := \big(pk_{(1-j)j}^{N-i}, pk_{1-j}^{(1)}, p_1^{N-i} + p_0^{N-i+1} + p_1^{N-i+1}\big)$.
2. U_1 computes $sk_{(1-j)j}^{(0),N-i} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(1),N-i}\big)$, $sk_{(1-j)j}^{N-i} = sk_{(1-j)j}^{(0),N-i} \oplus sk_{(1-j)j}^{(1),N-i}$, $\sigma_{rfd}^{(1),N-i} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{(1-j)j}^{N-i}, tx_{rfd}^{(1),N-i}\big)$, and posts $\big(tx_{rfd}^{(1),N-i}, \sigma_{rfd}^{(1),N-i}\big)$ on B_{1-j}.
3. After timeout T_0^{N-i}, if $i = 0$, U_0 generates $tx_{rfd}^{(0),N} := \big(pk_{j(1-j)}^{N}, pk_j^{(0)}, p_0^N + p_1^N\big)$; otherwise, U_0 generates $tx_{rfd}^{(0),N-i} := \big(pk_{j(1-j)}^{N-i}, pk_j^{(1)}, p_0^{N-i} + p_1^{N-i} + p_1^{N-i+1}\big)$.
4. U_0 computes $sk_{j(1-j)}^{(1),N-i} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(0),N-i}\big)$, $sk_{j(1-j)}^{N-i} = sk_{j(1-j)}^{(0),N-i} \oplus sk_{j(1-j)}^{(1),N-i}$, $\sigma_{rfd}^{(0),N-i} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{j(1-j)}^{N-i}, tx_{rfd}^{(0),N-i}\big)$, and posts $\big(tx_{rfd}^{(0),N-i}, \sigma_{rfd}^{(0),N-i}\big)$ on B_j.

After the timeout T_0^{N-i}, if $tx_{lock}^{(1),N-i}$ is not confirmed, U_0 can redeem its $(i+1)$-th round premiums and obtains U_1's i-th round premiums as compensation. If $i \geq 2$, after the timeout T_0^{N-i+1}, U_0 can also redeem its i-th round premium and obtains U_1's $(i-1)$-th round premiums as compensation.

1. After the timeout T_0^{N-i}, if $i = 0$, U_0 generates $tx_{rfd}^{(0),N-i} := \big(pk_{j(1-j)}^{N-i}, pk_j^{(0)}, p_0^{N-i} + p_1^{N-i}\big)$; otherwise, U_0 generates $tx_{rfd}^{(0),N-i} := \big(pk_{j(1-j)}^{N-i}, pk_j^{(0)}, p_0^{N-i} + p_1^{N-i} + p_1^{N-i+1}\big)$
2. U_0 computes $sk_{j(1-j)}^{(1),N-i} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(0),N-i}\big)$, $sk_{j(1-j)}^{N-i} = sk_{j(1-j)}^{(0),N-i} \oplus sk_{j(1-j)}^{(1),N-i}$, computes $\sigma_{rfd}^{(0),N-i} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{j(1-j)}^{N-i}, tx_{rfd}^{(0),N-i}\big)$, and posts $\big(tx_{rfd}^{(0),N-i}, \sigma_{rfd}^{(0),N-i}\big)$ on B_j.
3. If $i \geq 2$, after the timeout T_0^{N-i+1}, U_0 generates $tx_{rfd}^{(0),N-i+1} := \big(pk_{(1-j)j}^{N-i+1}, pk_j^{(0)}, p_0^{N-i+1} + p_1^{N-i+1} + p_1^{N-i+2}\big)$. Then, U_0 computes $sk_{(1-j)j}^{(1),N-i+1} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(0),N-i+1}\big)$, $sk_{(1-j)j}^{N-i+1} = sk_{(1-j)j}^{(0),N-i+1} \oplus sk_{(1-j)j}^{(1),N-i+1}$, computes $\sigma_{rfd}^{(0),N-i+1} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{(1-j)j}^{N-i+1}, tx_{rfd}^{(0),N-i+1}\big)$, and posts $\big(tx_{rfd}^{(0),N-i+1}, \sigma_{rfd}^{(0),N-i+1}\big)$ on B_{1-j}.

Swap Lock Phase

Users U_0 and U_1 lock their principals v_0 and v_1, respectively.

1. User U_0 computes $\sigma_{lock}^{(0),0} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_0^{(0)}, tx_{lock}^{(0),0}\big)$. It posts $\big(tx_{lock}^{(0)}, \big(\sigma_{lock}^{(0),0}, \sigma_{lock}^{(0),0'}\big)\big)$ on B_0.
2. User U_1 computes $\sigma_{lock}^{(1),0} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_1^{(1)}, tx_{lock}^{(1),0}\big)$. It posts $\big(tx_{lock}^{(1),0}, \big(\sigma_{lock}^{(1),0}, \sigma_{lock}^{(1),0'}\big)\big)$ on B_1.

Swap Complete Phase

User U_0 redeems U_1's v_1 and reclaim its $p_0^1 + p_1^1$, and then user U_1 redeems U_0's v_0 and reclaim its p_1^1.

1. User U_0 computes $\sigma_{swp}^{(0)} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{(0)}, y\big)$ and posts $\big(tx_{swp}^{(0)}, \sigma_{swp}^{(0)}\big)$ on B_1.
2. User U_1 computes $y \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Ext}\big(\sigma_{swp}^{(0)}, \tilde{\sigma}_{swp}^{(0)}, Y\big)$, and computes $\sigma_{swp}^{(1)} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{(1)}, y\big)$ and posts $\big(tx_{swp}^{(1)}, \sigma_{swp}^{(1)}\big)$ on B_0.

Swap Timeout Phase

After T_1^0, U_1 reclaims its principal v_1 and obtains U_0's $p_0^1 + p_1^1$, then U_0 reclaims its v_0 and obtains U_1's p_1^1 after T_0^0.

1. At time T_1^0, U_1 generates $tx_{rfd}^{(0),0} := \big(pk_{10}^0, pk_1^{(1)}, v_1 + p_0^1 + p_1^1\big)$, and computes $sk_{10}^{(0),0} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(1)}\big)$ and $sk_{10}^0 := sk_{10}^{(0),0} \oplus sk_{10}^{(1),0}$, $\sigma_{rfd}^{(0),0} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{10}^0, tx_{rfd}^{(0),0}\big)$. It posts $\big(tx_{rfd}^{(0),0}, \sigma_{rfd}^{(0),0}\big)$ on B_1.
2. At time T_0^0, U_0 generates $tx_{rfd}^{(0),0} := \big(pk_{01}^0, pk_0^{(0),0}, v_0 + p_1\big)$, and computes $sk_{01}^{(1),0} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C^{(0)}\big)$ and $sk_{01}^0 := sk_{01}^{(0),0} \oplus sk_{01}^{(1),0}$, $\sigma_{rfd}^{(0),0} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{01}^0, tx_{rfd}^{(0),0}\big)$. It posts $\big(tx_{rfd}^{(0),0}, \sigma_{rfd}^{(0),0}\big)$ on B_0.

Fig. 12: Round-based HedgeSwap (Party II)

Premium Setup Phase. Users U_0 and U_1 first create $2(N+1)$ shared addresses, pk_{01}^i on B_0 and pk_{10}^i on B_1, for each $i \in \{0, \ldots, N\}$. U_0 and U_1 generate pre-specific premium lock transactions, principal lock transactions, and swap transactions, and they then sign these transactions by 2PC. U_0 and U_1 also compute the coordinating to VTDs to ensure the honest party can redeem their coins and obtain counterparty premium as compensation. U_0 also generates $(Y, y) \in \mathcal{R}$ for atomic swap. Next, U_0 deposits $p_0^N + p_1^N$ by posting its $tx_{lock}^{(0),N}$, and U_1 deposits p_1^N by posting its $tx_{lock}^{(1),N}$. If either party aborts, the compliant party can refund its premium by opening its initial VTD.

Premium Round $i \in \{1, \ldots, N-1\}$. In premium round index i, U_0 deposits its $(i+1)$-th round premium, i.e., $p_0^{N-i} + p_1^{N-i}$, together with previously locked U_1's p_1^{N-i+1}, into pk_{01}^{N-i} (for $i \in \mathcal{R}_{\text{even}}$) or pk_{10}^{N-i} (for $i \in \mathcal{R}_{\text{odd}}$). Symmetrically, U_1 deposits p_1^{N-i}, together with previously locked U_0's $p_0^{N-i+1} + p_1^{N-i+1}$, into pk_{10}^{N-i} (for $i \in \mathcal{R}_{\text{even}}$ or pk_{01}^{N-i} (for $i \in \mathcal{R}_{\text{odd}}$). For $i \geq 2$, U_0 and U_1 can refund their $(i-1)$-th round premiums to their respective owners. The full details are shown in Figure 11.

Premium Timeout $i \in \{0, \ldots, N-1\}$. In premium round

TABLE I: Computation Time (ms).

Phase	SIG = Schnorr	SIG = ECDSA
Premium setup	432.790	439.568
Swap lock	1.072	1.520
Swap completed	0.844	1.132
Protocol	SIG = Schnorr	SIG = ECDSA
HedgeSwap	434.706	442.220
UAS [7]	428.455	433.107

TABLE II: Communication Overhead (bytes).

Protocol	SIG = Schnorr	SIG = ECDSA
HedgeSwap	577KB	581KB
UAS [7]	575KB	578KB

TABLE III: Gas Usage.

Phase	HedgeSwap	Contract-based solution [11]
Premium setup	319,047	932,658
Swap lock	40,979	28,487
Swap completed	21,000	65,172
Total	381,026	1,026,317

TABLE IV: Transaction Size (bytes).

Phase	HedgeSwap	HTLC-based solution [12]
Premium setup	192	355
Swap lock	341	541
Swap completed	192	336
Total	725	1232

index i, if U_0 aborts (i.e., $tx_{lock}^{(0),N-i}$ is not confirmed before T_1^{N-i+1}), U_1 can refund its premium p_1^{N-i+1}. For $i \geq 2$, U_1 can additionally obtain U_0's premium p_0^{N-i+1} as compensation. Similarly, if U_1 aborts (i.e., $tx_{lock}^{(1),N-i}$ is not confirmed before T_0^{N-i+1}), U_0 can refund its premium $p_0^{N-i+1}+p_1^{N-i+1}$ and $p_0^{N-i}+p_1^{N-i}$. For $i \geq 2$, U_0 obtains U_1's p_1^{N-i+2} as compensation for its locked $p_0^{N-i+1}+p_1^{N-i+1}$ and U_1's p_1^{N-i+1} as compensation for its locked $p_0^{N-i}+p_1^{N-i}$. The full details are shown in Figure 12.

Atomic Swap. Swap lock, swap complete and swap timeout phase are identical to the HedgeSwap, and the full details are shown in Figure 12.

The ideal functionality is deferred to Appendix B. A formal proof of the following theorem is provided in Appendix D.

Theorem 2. *If Σ_{AS} is a secure adaptor signature scheme w.r.t. a secure digital signature scheme Σ_{SIG} and a hard relation $\mathcal{R}$, Σ_{VTD} is a secure verifiable timed dlog scheme, Π_{SIG} and Π_{AS} are UC-secure 2PC protocols, then round-based HedgeSwap UC-realizes $\mathcal{F}_{\mathsf{RHSwap}}$ with access to the functionalities ($\mathcal{F}_{clock}$, $\mathcal{F}_{smt}$, $\mathcal{F}_B$).*

VII. Performance Evaluation

We implement prototypes of our HegdeSwap and round-based HegdeSwap protocols using C language to demonstrate the feasibility of our constructions and to evaluate its performance in terms of computation time, communication overhead, and on-chain costs. All experiments are conducted on a laptop equipped with an Ryzen AI 9 HX 370 2.00 GHz, 32 GB RAM, running Ubuntu. We instantiate Schnorr and ECDSA signatures over the *secp256k1* curve, and instantiate VTD with statistical parameters $n = 64$ and $t = n/2$ under a 1024-bit RSA modulus. We reuse the two-party computation (2PC) protocols of jointly key generation, signing, and pre-signing algorithms from [19]. The source code is available at [25].

Below we report the evaluation results of HedgeSwap and of round-based HedgeSwap with the number of rounds $N \in \{2, 3, 4, 5\}$.

Computation Time. We measure the computation time of our HedgeSwap and round-based HedgeSwap protocols. Table I shows that (i) HedgeSwap requires only 0.43 seconds for Schnorr-based instance and 0.44 seconds for ECDSA-based instance; (ii) the premium setup phase accounts for more than 99% of the total computation time, since users are required to compute the VTD instances and generate the signatures and pre-signatures for the corresponding transactions before locking coins. Compared to the basic universal atomic swap (UAS) [7], HedgeSwap increases only $6ms$ for Schnorr and $8ms$ for ECDSA, as HedgeSwap requires additional transactions and signature for users' premiums.

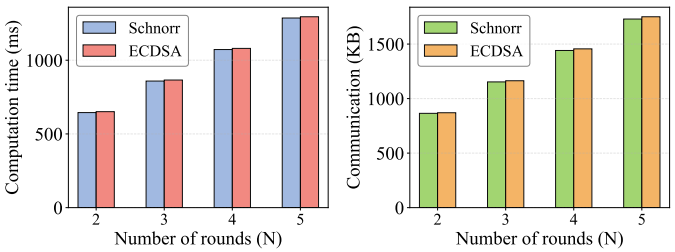
Figure 13a shows the computation time of round-based HedgeSwap under different values of N. It is observed that the time exhibit a linear increase as N grows, since each user generates and verifies $N+1$ VTDs and runs $N+1$ 2PC in our protocol. For example, when $N = 5$, the protocol requires $1286.273ms$ for Schnorr-based instance and $1294.788ms$ for ECDSA-based instance.

Communication Overhead. We measure the communication overhead based on the amount of messages exchanged between users. As shown in Table II, HedgeSwap requires 577KB for Schnorr and 581KB for ECDSA, primarily dominated by the VTD (about 288KB). Compared to UAS [7], HedgeSwap increases communication by only 2KB for Schnorr and 3KB for ECDSA. Figure 13b reports the communication overhead that exhibit a similar linear increase with N. Specifically, when $N = 5$, the protocol incurs 1729KB for Schnorr and 1752KB for ECDSA.

On-Chain costs. We evaluate the gas usage of HedgeSwap per phase on the Ethereum Sepolia testnet and compare it with the contract-based hedged swap protocol in [11]. The results are summarized in Table III. Specifically, HedgeSwap requires (i) a premium lock transaction costing 319,047 gas (including contract creation for the MIMO address), (ii) a principal lock transaction that transfers principal and premium into a shared address, costing 40,979 gas, and (iii) a swap transaction with the standard cost of 21,000 gas. Overall, HedgeSwap consumes 381,026 gas, compared to 1,026,317 gas for the contract-based solution [11], achieving a $2.69\times$ reduction and saving about $2.18 USD (based on the exchange rate in Dec. 2025).

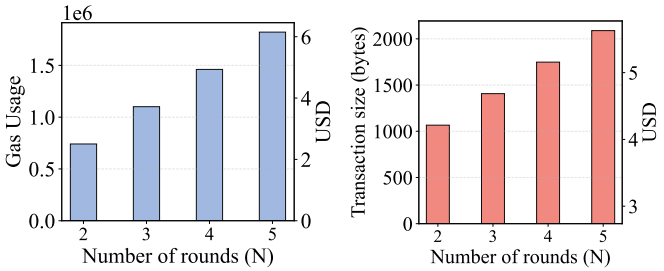
We further measure transaction sizes on the Bitcoin testnet and compare them with the HTLC-based method [12]. As reported in Table IV, HedgeSwap has a total transaction size of 725 bytes, achieving a $1.70\times$ reduction (saving about $1.41 USD) compared to the HTLC-based solution [12].

Figure 14a and 14b show the on-chain costs of round-based HedgeSwap in terms of gas usage and transaction size, respectively. Specifically, when $N = 5$, the protocol consumes



(a) Computation time. (b) Communication overhead.

Fig. 13: Off-chain costs of round-based HedgeSwap.



(a) Gas usage. (b) Transaction size.

Fig. 14: On-chain costs of round-based HedgeSwap.

1,821,130 gas (∼$6.13 USD) on the Sepolia testnet and has a total transaction size of 2,089 bytes (∼$5.79 USD) on the Bitcoin testnet.

VIII. Related Work

Thyagarajan et al. [7] introduced the first universal atomic swap protocol by utilizing adaptor sigantures and VTD instead of HTLC scripts. Subsequently, Gökay et al. [26] developed an atomic swap protocol compatible with BLS-based blockchains. Hanzlik et al. [18] enabled users to swap assets with an untrusted intermediary unlinkably and atomically. Ni et al. [19] proposed PipeSwap, which introduces a two-hop completion mechanism to address the timeout race attacks in basic universal atomiuc swaps. Xiao et al. [22] presented universal parallel multi-party swaps by treating every user as a leader. However, these approaches remain vulnerable to griefing attacks.

Most cross-chain swap protocols employed premium-based mechanisms to mitigate griefing attacks. Han et al. [10] quantified the unfairness of optionality and proposed that Alice pays a premium to Bob if she deviates from the protocol. Xue and Herlihy [11] introduced a hedged swap protocol that compensates conforming parties with the deviating party's premium. Nadahalli et al. [12] implemented a griefing-free two-party swap using only HTLC scripts. Singh et al. [13] further achieved a grief-free swap by just four on-chain transactions. Mazumdar [14] proposed a quick swap, enabling a compliant party to claim compensation from a deviating party before the timeout. Zhuo et al. [15] introduced a fast settlement protocol that incentivizes a deviating party to promptly refund the counterparty's locked assets. Kai et al. [16] proposed a credit mechanism that adjusts premium amounts based on participants' reputations. Unfortunately, these solutions rely on HTLC scripts or smart contracts, making them incompatible with scriptless blockchains.

Robinson [27] proposed packetized payments, in which a transaction is split into a series of minimal swaps to reduce the impact of griefing, but this method is feasible only for fungible and divisible cryptocurrencies. Some swap protocols assume the presence of a centralized entity that will not engage in griefing attacks. Arwen protocol [17] relied on a centralized exchange incentivized by its revenue and reputation to avoid malicious behavior. Hanzlik et al. [18] used the registration protocol [28], which ensures that a centralized exchange locks assets only when a user has locked an equal amount.

IX. Conclusion

In this paper, we first propose HedgeSwap, a universal hedged atomic swap protocol against griefing attacks in atomic swaps, relying only on signature verification from the underlying blockchains. We further propose a round-based HedgeSwap for high-value asset swaps. We identify timeout race attacks in universal swap protocols and the timeout overlap dilemma in premium-based mechanisms. To address these issues, we eliminate the premium timeout in HedgeSwap and introduce a premium migration mechanism for each round in the round-based HedgeSwap. Moving forward, we plan to extend universal hedged atomic swaps to multi-party swap settings.

References

[1] G. Wang, Q. Wang, and S. Chen, "Exploring blockchains interoperability: A systematic survey," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–38, 2023.

[2] A. Augusto, R. Belchior, M. Correia, A. Vasconcelos, L. Zhang, and T. Hardjono, "Sok: Security and privacy of blockchain interoperability," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 3840–3865.

[3] T. Nolan, "Alt chains and atomic transfers," https://bitcointalk.org/index.php?topic=193281.0, May 2013, bitcoin Forum.

[4] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 459–474.

[5] R. W. Lai, V. Ronge, T. Ruffing, D. Schröder, S. A. K. Thyagarajan, and J. Wang, "Omniring: Scaling private payments without trusted setup," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 31–48.

[6] D. Schwartz, N. Youngs, A. Britto *et al.*, "The ripple protocol consensus algorithm," *Ripple Labs Inc White Paper*, vol. 5, no. 8, p. 151, 2014.

[7] S. A. Thyagarajan, G. Malavolta, and P. Moreno-Sanchez, "Universal atomic swaps: Secure exchange of coins across all blockchains," in *2022 IEEE symposium on security and privacy (SP)*. IEEE, 2022, pp. 1299–1316.

[8] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized channels from limited blockchain scripts and adaptor signatures," in *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part II 27*. Springer, 2021, pp. 635–664.

[9] S. A. K. Thyagarajan, A. Bhat, G. Malavolta, N. Döttling, A. Kate, and D. Schröder, "Verifiable timed signatures made practical," in *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, 2020, pp. 1733–1750.

[10] R. Han, H. Lin, and J. Yu, "On the optionality and fairness of atomic swaps," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, 2019, pp. 62–75.

[11] Y. Xue and M. Herlihy, "Hedging against sore loser attacks in cross-chain transactions," in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, 2021, pp. 155–164.
[12] T. Nadahalli, M. Khabbazian, and R. Wattenhofer, "Grief-free atomic swaps," in *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2022, pp. 1–9.
[13] K. Singh, V. J. Ribeiro, and S. Mandal, "4-swap: Achieving grief-free and bribery-safe atomic swaps using four transactions," in *7th Conference on Advances in Financial Technologies (AFT 2025)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025, pp. 32–1.
[14] S. Mazumdar, "Towards faster settlement in htlc-based cross-chain atomic swaps," in *2022 IEEE 4th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2022, pp. 295–304.
[15] F. Zhuo, Z. Song, L. Jia, H. Zhang, Z. Li, and Y. Sun, "Enabling fast settlement in atomic cross-chain swaps," in *International Conference on Security and Privacy in Communication Systems*. Springer, 2023, pp. 395–412.
[16] K. Wang, D. Wang, H. Zhi, Y. Chen, and X. Zhang, "Hash time lock with dynamic premium based on credit in cross-chain transaction," in *2024 IEEE International Conference on Blockchain (Blockchain)*, 2024, pp. 123–130.
[17] E. Heilman, S. Lipmann, and S. Goldberg, "The arwen trading protocols," in *Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers 24*. Springer, 2020, pp. 156–173.
[18] L. Hanzlik, J. L. Sri, A. Thyagarajan, and B. Wagner, "Sweep-uc: Swapping coins privately," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3822–3839.
[19] P. Ni, A. Tian, and J. Xu, " Warning! The Timeout T Cannot Protect You From Losing Coins PipeSwap: Forcing the Timely Release of a Secret for Atomic Cross-Chain Swaps ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, 2025, pp. 1566–1583.
[20] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. IEEE, 2001, pp. 136–145.
[21] R. Canetti, Y. Dodis, R. Pass, and S. Walfish, "Universally composable security with global setup," in *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007. Proceedings 4*. Springer, 2007, pp. 61–85.
[22] D. Xiao, C. Zhang, H. Deng, J. Liang, L. Wang, and L. Zhu, "Parallelizing universal atomic swaps for multi-chain cryptocurrency exchanges," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 4073–4092.
[23] J. C. Cox, S. A. Ross, and M. Rubinstein, "Option pricing: A simplified approach," *Journal of financial Economics*, vol. 7, no. 3, pp. 229–263, 1979.
[24] J. Katz, U. Maurer, B. Tackmann, and V. Zikas, "Universally composable synchronous computation," in *Theory of Cryptography Conference*. Springer, 2013, pp. 477–498.
[25] Anonymous, "Hedgeswap code," Anonymous GitHub, https://anonymous.4open.science/r/HedgeSwap-1/.
[26] H. Gokay, F. Baldimtsi, and G. Ateniese, "Atomic swaps for boneh–lynn–shacham (bls) based blockchains," in *European Symposium on Research in Computer Security*. Springer, 2024, pp. 341–361.
[27] D. Robinson, "Htlcs considered harmful," 2019. [Online]. Available: http://diyhpl.us/wiki/transcripts/stanford-blockchain-conference/2019/htlcs-considered-harmful/
[28] E. Tairi, P. Moreno-Sanchez, and M. Maffei, "A 2 l: Anonymous atomic locks for scalability in payment channel hubs," in *2021 IEEE symposium on security and privacy (SP)*. IEEE, 2021, pp. 1834–1851.

APPENDIX A
BUILDING BLOCKS

A. Adaptor Signature

Adaptor signatures allow users to generate an encrypted signature (pre-signature) on a message m, which is not valid by itself but can be adapted into a valid signature if the user knows a certain witness. Here, we give a formal description of adaptor signatures, following the definitions and security experiments from [8].

Definition 1 (Adaptor Signature)**.** *An adaptor signature w.r.t. a signature scheme* $\Sigma_{\mathsf{SIG}} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ *and a hard relation* $\mathcal{R}$*, comprising four algorithms* $\Sigma_{\mathsf{AS}} = (\mathsf{pSign}, \mathsf{pVrfy}, \mathsf{Adapt}, \mathsf{Ext})$.

- $\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$*: It is a PPT algorithm that on input a private key* sk*, a message* $m \in \{0,1\}^*$*, and a puzzle* $Y \in L_{\mathcal{R}}$*, outputs a pre-signature* $\tilde{\sigma}$.
- $0/1 \leftarrow \mathsf{pVrfy}(m, pk, Y, \tilde{\sigma})$ *: It is a DPT algorithm that, on input, a message* $m \in \{0,1\}^*$*, a public key* pk*, and a puzzle* $Y \in L_{\mathcal{R}}$*, and a pre-signature* $\tilde{\sigma}$*, outputs a bit* $0/1$.
- $\sigma \leftarrow \mathsf{Adapt}(\tilde{\sigma}, y)$ *: It is a DPT algorithm that, on input, a pre-signature* $\tilde{\sigma}$ *and a witness* y*, outputs a valid signature* σ.
- $y \leftarrow \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$ *: It is a DPT algorithm that, on input, a signature* σ*, a pre-signature* $\tilde{\sigma}$*, and a puzzle* $Y \in L_{\mathcal{R}}$*, outputs a witness* y *such that* $(Y, y) \in \mathcal{R}$.

Definition 2 (Pre-signature Correctness)**.** *An adaptor signature scheme* Σ_{AS} *satisfies pre-signature correctness stating that for every* $m \in \{0,1\}^*$ *and each* $(Y, y) \in \mathcal{R}$*, the following condition is satisfied:*

$$\Pr\left[\begin{array}{c} \mathsf{pVrfy}(pk, m, Y, \tilde{\sigma}) = 1 \\ \wedge \mathsf{Vrfy}(pk, m, \sigma) = 1 \\ \wedge (Y, y) \in \mathcal{R} \end{array} : \begin{array}{c} (pk, sk) \leftarrow \mathsf{KGen}(1^\lambda) \\ \tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y) \\ \sigma := \mathsf{Adapt}(\tilde{\sigma}, y) \\ y := \mathsf{Ext}(\sigma, \tilde{\sigma}, Y) \end{array}\right] = 1.$$

Definition 3 (aEUF-CMA security)**.** *An adaptor signature scheme is unforgeable if, for any PPT adversary* $\mathcal{A}$*, there exists a negligible function* ν *such that* $\Pr[\mathrm{aSigForge}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda) = 1] \leq \nu(\lambda)$*, where the experiment* $\mathit{aSigForge}_{\mathcal{A},\Sigma_{\mathsf{AS}}}$ *is defined in Figure 15.*

Definition 4 (Pre-signature Adaptability)**.** *An adaptor signature scheme satisfies pre-signature adaptability if for any message* $m \in \{0,1\}^*$*, any statement/witness pair* $(Y, y) \in \mathcal{R}$*, any public key* pk*, and any pre-signature* $\tilde{\sigma} \in \{0,1\}^*$ *with* $\mathsf{pVrfy}(pk, m, Y, \tilde{\sigma}) = 1$*, we have* $\mathsf{Vrfy}(pk, m, \mathsf{Adapt}(\tilde{\sigma}, y)) = 1$.

Definition 5 (Witness Extractability)**.** *An adaptor signature scheme is witness extractable if, for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\Pr[\mathrm{aWitExt}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda) = 1] \leq \nu(\lambda)$*, where the experiment* $\mathrm{aWitExt}_{\mathcal{A},\Sigma_{\mathsf{AS}}}$ *is defined in Figure 16.*

B. Verified Timed Dlog

A verifiable timed dlog (discrete logarithm) [9], [7] is defined w.r.t. a group $\mathbb{G}$ of order q and a generator g. Here, a committer generates a timed commitment with timing hardness T for a secret $sk \in \mathbb{Z}_q$ such that $pk = g^{sk}$ where pk is public and sk is referred to as the dlog (discrete logarithm) of pk (w.r.t. g). The verifier checks the well-formedness of the timed commitment and is able to learn the value sk by forcibly opening the commitment within time T. The formal definition is as follows.

$\underline{\mathrm{aSigForge}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda)}$
$\mathcal{Q} := \emptyset$
$(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$
$(Y, y) \leftarrow \mathsf{RGen}(1^\lambda)$
$m \leftarrow \mathcal{A}^{\mathcal{O}_{\mathrm{S}}(\cdot),\mathcal{O}_{\mathrm{pS}}(\cdot,\cdot)}(pk, Y)$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\sigma \leftarrow \mathcal{A}^{\mathcal{O}_{\mathrm{S}}(\cdot),\mathcal{O}_{\mathrm{pS}}(\cdot,\cdot)}(\tilde{\sigma})$
return $m \notin \mathcal{Q} \wedge \mathsf{Vrfy}(pk, m, \sigma)$

$\underline{\mathcal{O}_{\mathrm{S}}(m)}$
$\sigma \leftarrow \mathsf{Sign}(sk, m)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return σ
$\underline{\mathcal{O}_{\mathrm{pS}}(m, Y)}$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return $\tilde{\sigma}$

Fig. 15: Unforgeabiltiy Experiment of AS

$\underline{\mathrm{aWitExt}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda)}$
$\mathcal{Q} := \emptyset$
$(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$
$(m, Y) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathrm{S}}(\cdot),\mathcal{O}_{\mathrm{pS}}(\cdot,\cdot)}(pk)$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\sigma \leftarrow \mathcal{A}^{\mathcal{O}_{\mathrm{S}}(\cdot),\mathcal{O}_{\mathrm{pS}}(\cdot,\cdot)}(\tilde{\sigma})$
$y' := \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$
return $m \notin \mathcal{Q} \wedge (Y, y') \notin \mathcal{R}$ $\wedge \mathsf{Vrfy}(pk, m, \sigma)$

$\underline{\mathcal{O}_{\mathrm{S}}(m)}$
$\sigma \leftarrow \mathsf{Sign}(sk, m)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return σ
$\underline{\mathcal{O}_{\mathrm{pS}}(m, Y)}$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return $\tilde{\sigma}$

Fig. 16: Witness Extractability Experiment of AS

Definition 6 (Verifiable Timed Dlog)**.** *A verifiable timed dlog is a tuple of four algorithms* $\Sigma_{\mathsf{VTD}} = (\mathsf{Commit}, \mathsf{Verify}, \mathsf{Open}, \mathsf{ForceOp})$ *as follow:*

- $(C, \pi) \leftarrow \mathsf{Commit}(sk, T)$: *the commit algorithm takes a discrete log value* $sk \in \mathbb{Z}_q$ *(generated using* $\mathsf{SIG.KGen}(1^\lambda)$*), a hiding time* T *and outputs a commitment* C *and a proof* π*.*
- $0/1 \leftarrow \mathsf{Verify}(pk, C, \pi)$: *the verify algorithm takes a public key* pk*, a commitment* C *of hardness* T*, and a proof* π *and accepts the proof by outputting 1 if and only if, the value* sk *embedded in* C *is a valid commitment. Otherwise, it outputs 0.*
- $(sk, r) \leftarrow \mathsf{Open}(C)$: *the open phase where the committer takes a commitment* C *and outputs the committed value* sk *and the randomness* r *used in generating* C*.*
- $sk \leftarrow \mathsf{ForceOp}(C)$: *the force open algorithm takes the commitment* C *and outputs the value* sk*.*

Definition 7 (Soundness)**.** *A VTD is sound if there is a negligible function* $\nu(\cdot)$ *such that for all PPT adversaries* $\mathcal{A}$ *and all* $\lambda \in \mathbb{N}$*, the probability that the verification fails is bounded by:*

$$\Pr\left[\, b_1 = 1 \wedge b_2 = 0 \;\middle|\; \begin{array}{l} (pk, C, \pi, T) \leftarrow \mathcal{A}(1^\lambda), \\ sk \leftarrow \mathsf{ForceOp}(C), \\ b_1 := \mathsf{Verify}(pk, C, \pi), \\ b_2 := (pk = g^{sk}) \end{array} \right] \leq \nu(\lambda).$$

Definition 8 (Privacy)**.** *A VTD is private if there exists a PPT simulator* $\mathcal{S}$*, a negligible function* $\nu(\cdot)$*, and a polynomial* $\tilde{T}$*, such that for all polynomials* $T > \tilde{T}$*, all PRAM algorithms* $\mathcal{A}$ *running within a time* $t < T$*, and all* $\lambda \in \mathbb{N}$ *it holds that*

$$\left| \begin{array}{l} \Pr\left[\mathcal{A}(pk, C, \pi) = 1 \;\middle|\; \begin{array}{l} sk \leftarrow \{0,1\}^\lambda, \\ pk := g^{sk} \\ (C, \pi) \leftarrow \mathsf{Commit}(sk, T) \end{array} \right] \\ - \Pr\left[\mathcal{A}(pk, C, \pi) = 1 \;\middle|\; \begin{array}{l} sk \leftarrow \{0,1\}^\lambda, \\ pk := g^{sk} \\ (C, \pi) \leftarrow \mathsf{Sim}(pk, T) \end{array} \right] \end{array} \right| \leq \nu(\lambda)$$

Appendix B
Formal Functionality for Round-based HedgeSwap

We formalize the security of our round-based HedgeSwap protocol using an ideal functionality $\mathcal{F}_{\mathsf{RHSwap}}$, which is illustrated in Figure 18.

The functionality $\mathcal{F}_{\mathsf{RHSwap}}$ interacts with two users U_0 and U_1, a simulator $\mathcal{S}$, and two global blockchain functionalities $\mathcal{F}_{B_0}$ and $\mathcal{F}_{B_1}$. The functionality is also parameterized by timeouts T_0^i and T_1^i for each $i \in [0, N]$. It maintains a list Locked that stores the locked coins in the functionality, initialized to $\emptyset$. It also maintains state variables b_j^i for $j \in \{0, 1\}$ and $i \in [0, N]$, representing the lock state of user U_j's coins in round index i. All these variables are initialized to $\perp$. The functionality specifies four procedures as follows, each triggered by a message that includes a respective request and a session identifier sid.

Premium Setup. User U_0 sends a message (pre-lock, sid, $pk_j^{(0)}$, $sk_j^{(0)}$, $p_0^N + p_1^N$) to initiate the first-round premium phase, where $j = 0$ if N is even and $j = 1$ otherwise. $\mathcal{F}_{\mathsf{RHSwap}}$ transfers coins $p_0^N + p_1^N$ to a specific account $pk_{\mathcal{F},0}^N$ for a time period T_0^N. After time T_0^N, if the coins $p_0^N + p_1^N$ are still locked, then $\mathcal{F}_{\mathsf{RHSwap}}$ refunds $p_0^N + p_1^N$ to $pk_j^{(0)}$.

User U_1 also sends a message (pre-lock, sid, $pk_j^{(1)}$, $sk_j^{(1)}$, p_1^N) to $\mathcal{F}_{\mathsf{RHSwap}}$, where $j = 1$ if N is even and $j = 0$ otherwise. $\mathcal{F}_{\mathsf{RHSwap}}$ transfers p_1^N to $pk_{\mathcal{F},1}^N$ for a time period T_1^N. After time T_1^N, if these coins are still locked, then $\mathcal{F}_{\mathsf{RHSwap}}$ refunds p_1^N to $pk_j^{(1)}$.

Premium Lock Round Index $i \in \{1, \ldots, N-1\}$**.** User U_0 sends a premium lock message (pre-lock-i, sid, $pk_j^{(0)}$, $sk_j^{(0)}$, $p_0^{N-i}+p_1^{N-i}$) at time t to specify the $(i+1)$-th round premium $p_0^{N-i}+p_1^{N-i}$, where $j = 0$ if $(N-i)$ is even and $j = 1$ otherwise. If $b_1^{N-i+1} = 1$ and $t < T_1^{N-i+1}$, then $\mathcal{F}_{\mathsf{RHSwap}}$ transfers $p_0^{N-i}+p_1^{N-i}$ together with p_1^{N-i+1}, previously locked in $pk_{\mathcal{F},0}^{N-i+1}$, to $pk_{\mathcal{F},0}^{N-i}$ for a time period T_0^{N-i}; otherwise, it aborts. For $i \geq 2$, it also refunds $p_0^{N-i+2}+p_1^{N-i+2}$ to $pk_j^{(0)}$. After time T_0^{N-i}, if the coins are still locked, then $\mathcal{F}_{\mathsf{RHSwap}}$ refunds them to $pk_j^{(0)}$.

Similarly, U_1 sends (pre-lock-i, sid, $pk_j^{(1)}$, $sk_j^{(1)}$, p_1^{N-i}) at time t to specify the $(i+1)$-th round premium p_1^{N-i}, where $j = 1$ if $(N-i)$ is even and $j = 0$ otherwise. If $b_0^{N-i} = 1$ and $t < T_0^{N-i+1}$, then $\mathcal{F}_{\mathsf{RHSwap}}$ transfers p_1^{N-i} together with $p_0^{N-i+1}+p_0^{N-i+1}$, previously locked in $pk_{\mathcal{F},1}^{N-i+1}$, to $pk_{\mathcal{F},1}^{N-i}$ for a time T_1^{N-i}; otherwise, it aborts. For $i \geq 2$, it also refunds p_1^{N-i+2} to $pk_j^{(1)}$. After time T_1^{N-i}, if the coins are still locked, then $\mathcal{F}_{\mathsf{RHSwap}}$ refunds them to $pk_j^{(1)}$.

Premium Setup Phase

- Upon receiving $(\text{pre-lock}, sid, pk_j^{(0)}, sk_j^{(0)}, p_0^N + p_1^N)$ from U_0, where $j = 0$ if N is even and $j = 1$ otherwise:
  1. Invoke the subroutine $\mathsf{Freeze}(sid, pk_j^{(0)}, pk_{\mathcal{F},0}^N, p_0^N + p_0^N, T_0^N)$.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_j}$, append $(sid, pk_j^{(0)}, p_0^N + p_1^N, T_0^N)$ to Locked, set $b_0^N = 1$, and send (frz, ok) to U_0 and U_1.
  3. Upon receiving $(\mathsf{pre\text{-}rfd}, sid, pk_j^{(0)})$ from U_0 at time t', if $t' > T_0^N$ and the corresponding entry is still in Locked, then invoke the subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F},0}^N, pk_j^{(0)}, p_0^N + p_0^N)$, delete it from Locked, and set $b_0^N = 0$.
- Upon receiving $(\text{pre-lock}, sid, pk_j^{(1)}, sk_j^{(1)}, p_1^N)$ from U_1, where $j = 1$ if N is even and $j = 0$ otherwise,
  1. If $b_0^N = 1$, invoke subroutine $\mathsf{Freeze}(sid, pk_j^{(0)}, pk_{\mathcal{F},1}^N, p_1^N, T_1^N)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_j}$, append $(sid, pk_j^{(1)}, p_1^N, pk_{\mathcal{F},1}^N, T_1^N)$ to Locked, set $b_1^N = 1$, and send (frz, ok) to U_0 and U_1.
  3. Upon receiving $(\mathsf{pre\text{-}rfd}, sid, pk_j^{(1)})$ from U_1 at time t', if $t' > T_1^N$ and the corresponding entry is still in Locked, then invoke the subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F},1}^N, pk_j^{(1)}, p_1^N)$, delete it from Locked, and set $b_1^N = 0$.

Premium Lock Round Index $i \in \{1, \ldots, N-1\}$ Phase

- Upon receiving $(\text{pre-lock-}i, sid, pk_j^{(0)}, sk_j^{(0)}, p_0^{N-i}+p_1^{N-i})$ from U_0 at time t, where $j = 0$ if $(N-i)$ is even and $j = 1$ otherwise,
  1. If $b_1^{N-i+1} = 1 \wedge t < T_1^{N-i+1}$, invoke subroutine $\mathsf{Freeze}(sid, (pk_j^{(0)}, pk_{\mathcal{F},1}^{N-i+1}), pk_{\mathcal{F},0}^{N-i}, p_0^{N-i}+p_1^{N-i}+p_1^{N-i+1})$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_j}$, append $(sid, pk_j^{(i)}, p_0^{N-i}+p_1^{N-i}+p_1^{N-i+1}, pk_{\mathcal{F},0}^{N-i}, T_0^{N-i})$ to Locked, set $b_0^{N-i} = 1$, and sends (frz, ok) to U_0 and U_1.
  3. If $i = 1$, delete the entry $(sid, pk_j^{(1)}, p_1^N, pk_{\mathcal{F},1}^N, T_1^N)$ from the list Locked.
  4. If $i \geq 2$, delete the entry $(sid, pk_j^{(1)}, p_1^{N-i+1}+p_0^{N-i+2}+p_1^{N-i+2}, pk_{\mathcal{F},1}^{N-i+1}, T_1^{N-i+1})$ and invoke $\mathsf{UnFreeze}(sid, pk_{\mathcal{F},1}^{N-i+1}, pk_j^{(0)}, p_0^{N-i+2}+p_1^{N-i+2})$.
  5. Upon receiving $(\mathsf{rfd}\text{-}i, sid, pk_j^{(0)})$ from U_0 at time t', if $t' > T_0^{N-i}$ and this entry $(sid, pk_j^{(i)}, p_0^{N-i}+p_1^{N-i}+p_1^{N-i+1}, pk_{\mathcal{F},0}^{N-i}, T_0^{N-i})$ is still in Locked, then invoke the subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F},0}^{N-i}, pk_j^{(0)}, p_0^{N-i}+p_1^{N-i}+p_1^{N-i+1})$, delete the entry from Locked, and set $b_0^{N-i} = 0$.
- Upon receiving $(\text{pre-lock-}i, sid, pk_j^{(1)}, sk_j^{(1)}, p_1^{N-i})$ from U_1 at time t, where $j = 1$ if $(N-i)$ is even and $j = 0$ otherwise,
  1. If $b_0^{N-i} = 1 \wedge t < T_0^{N-i+1}$, invoke subroutine $\mathsf{Freeze}(sid, (pk_j^{(1)}, pk_{\mathcal{F},0}^{N-i+1}), pk_{\mathcal{F},1}^{N-i}, p_1^{N-i}+p_0^{N-i+1}+p_1^{N-i+1})$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_j}$, append $(sid, pk_j^{(i)}, p_1^{N-i}+p_0^{N-i+1}+p_1^{N-i+1}, pk_{\mathcal{F},1}^{N-i}, T_1^{N-i})$ to Locked, set $b_1^{N-i} = 1$, and sends (frz, ok) to U_0 and U_1.
  3. If $i = 1$, delete the entry $(sid, pk_j^{(0)}, p_0^N+p_1^N, pk_{\mathcal{F},0}^N, T_0^N)$ from the list Locked.
  4. If $i \geq 2$, delete the entry $(sid, pk_j^{(0)}, p_0^{N-i+1}+p_1^{N-i+1}+p_1^{N-i+2}, pk_{\mathcal{F},0}^{N-i+1}, T_0^{N-i+1})$ from Locked, and invoke $\mathsf{UnFreeze}(sid, pk_{\mathcal{F},0}^{N-i+1}, pk_j^{(1)}, p_1^{N-i+2})$.
  5. Upon receiving $(\mathsf{rfd}\text{-}i, sid, pk_j^{(1)})$ from U_1 at time t', if $t' > T_1^{N-i}$ and this entry $(sid, pk_j^{(i)}, p_1^{N-i}+p_0^{N-i+1}+p_1^{N-i+1}, pk_{\mathcal{F},1}^{N-i}, T_1^{N-i})$ is still in Locked, then invoke the subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F},1}^{N-i}, pk_j^{(1)}, p_1^{N-i}+p_0^{N-i+1}+p_1^{N-i+1})$, delete the entry from Locked, and set $b_0^{N-i} = 0$.

Fig. 17: Ideal Functionality $\mathcal{F}_{\mathsf{RHSwap}}$ (Part I)

Swap Lock. User U_0 sends $(\mathsf{lock}, sid, pk_0^{(0)}, sk_0^{(0)}, v_0)$ at time t to specify that the swap principal amount v_0. If $b_1^1 = 1$ and $t < T_1^1$, then $\mathcal{F}_{\mathsf{RHSwap}}$ transfers v_0 together with p_1^1, locked in $pk_{\mathcal{F},0}^1$, to $pk_{\mathcal{F},0}^0$ for a time period T_0^0, and refunds $p_0^2+p_1^2$ to $pk_0^{(0)}$; otherwise, it aborts. After time T_0^0, if the coins are still locked, then $\mathcal{F}_{\mathsf{RHSwap}}$ refunds $v_0 + p_1^1$ to $pk_0^{(0)}$.

Similarly, U_1 sends $(\mathsf{lock}, sid, pk_1^{(1)}, sk_1^{(1)}, v_1)$ at time t to specify that the principal amount v_1. If $b_0^0 = 1$ and $t < T_1^0$, then $\mathcal{F}_{\mathsf{RHSwap}}$ transfers v_1 together with $p_0^1 + p_1^1$, locked in $pk_{\mathcal{F},1}^1$, to $pk_{\mathcal{F},1}^0$ for a time period T_1^0 and refunds p_1^2 to $pk_1^{(1)}$; otherwise, it aborts. After time T_1^0, if the coins are still locked, then $\mathcal{F}_{\mathsf{RHSwap}}$ refunds $v_1 + p_0^1 + p_1^1$ to $pk_1^{(1)}$.

Swap Complete. User U_0 sends the swap message $(\mathsf{swp}, sid, pk_1^{(0)})$ at time t. If $b_1^0 = 1$ and $t < T_1^0$, $\mathcal{F}_{\mathsf{RHSwap}}$ transfers coins $v_1 + p_0^1 + p_1^1$ to $pk_1^{(0)}$; otherwise, it aborts. Upon receiving the swap message $(\mathsf{swp}, sid, pk_0^{(1)})$ from U_1, if $b_0 = 1$, $\mathcal{F}_{\mathsf{RHSwap}}$ transfers coins $v_0 + p_1^1$ to $pk_0^{(1)}$; otherwise, it aborts.

APPENDIX C
PROOF OF THEOREM 1

Proof of Theorem 1. We now prove that HedgeSwap protocol UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{HSwap}}$.

To show the indistinguishability between the ideal world and the real world, we construct a simulator $\mathcal{S}$ to simulate the protocol in the real world while interacting with the ideal functionality $\mathcal{F}$. At the beginning, $\mathcal{S}$ corrupts one user of $\{U_0, U_1\}$ as $\mathcal{A}$ does. We begin with the real world protocol execution, gradually change the simulation in these hybrids and then we argue about the proximity of neighbouring experiments.

Hybrid $\mathcal{H}_0$: It is the same as the real world protocol execution;

Hybrid $\mathcal{H}_1$: It is the same as the above execution except that the 2PC protocol Π_{SIG} in the premium setup phase to generate

Swap Lock Phase

- Upon receiving $(\mathsf{lock}, sid, pk_0^{(0)}, pk_0^{(1)}, v_0)$ from U_0 at time t,
  1. If $b_1^1 = 1 \wedge t < T_1^1$, invoke subroutines $\mathsf{Freeze}(sid, (pk_0^{(0)}, pk_{\mathcal{F},1}^1), pk_{\mathcal{F},0}^0, v_0{+}p_1^1, T_0^0)$ and $\mathsf{UnFreeze}(sid, pk_{\mathcal{F},1}^1, pk_0^{(0)}, p_0^2 + p_1^2)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_0}$, delete the entry $(sid, pk_0^{(1)}, p_1^1{+}p_0^2{+}p_1^2, pk_{\mathcal{F},1}^1, T_1^1)$, append this entry $(sid, pk_0^{(0)}, v_0{+}p_1, pk_{\mathcal{F},1}^1, T_0^0)$ to the list Locked, set $b_0^0 = 1$, and send (frz, ok) to U_0 and U_1.
  3. Upon receiving $(\mathsf{rfd}, sid, pk_j^{(0)})$ from U_0 at time t', if $t' > T_0^0$ and this entry $(sid, pk_0^{(0)}, v_0{+}p_1, pk_{\mathcal{F},0}^0, T_0^0)$ is still in Locked, then invoke subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F},0}^0, pk_0^{(0)}, v_0{+}p_1)$, delete the entry from Locked, and set $b_0^0 = 1$.
- Upon receiving $(\mathsf{lock}, sid, pk_1^{(1)}, sk_1^{(1)}, v_1)$ from U_1 at time t,
  1. if $b_0^0 = 1 \wedge t < T_0^1$, invoke subroutines $\mathsf{Freeze}(sid, (pk_1^{(0)}, pk_{\mathcal{F},0}^1), pk_{\mathcal{F},1}^0, v_1 + p_0^1 + p_1^1, T_1^0)$ and $\mathsf{UnFreeze}(sid, pk_{\mathcal{F},1}^1, pk_1^{(1)}, p_1^2)$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_1}$, delete the entry $(sid, pk_1^{(0)}, p_0^1{+}p_0^1{+}p_1^2, pk_{\mathcal{F},0}^1, T_0^1)$, append the entry $(sid, pk_1^{(1)}, v_1 + p_0 + p_1, pk_{\mathcal{F},1}^0, T_1^0)$ to the list Locked, set $b_1^0 = 1$, and send (frz, ok) to U_0 and U_1.
  3. Upon receiving $(\mathsf{rfd}, sid, pk_j^{(1)})$ from U_1 at time t', if $t' > T_1^0$ and this entry $(sid, pk_1^{(1)}, v_1{+}p_0{+}p_1, pk_{\mathcal{F},1}^0, T_1^0)$ is still in Locked, then invoke subroutine $\mathsf{Unfreeze}(sid, pk_{\mathcal{F}}^1, pk_1^{(1)}, v_1 + p_0^1 + p_1^1)$, delete the entry from Locked, and set $b_1^0 = 1$.

Swap Complete

- Upon receiving $(\mathsf{swp}, sid, pk_1^{(0)})$ from U_0 at time t,
  1. If $b_1^0 = 1 \wedge t < T_1^0$, invoke subroutine $\mathsf{Transfer}(id, pk_{\mathcal{F},1}^0, pk_1^{(0)}, v_1 + p_0 + p_1)$, and set $b_1^0 = 0$; otherwise, abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_1}$, delete the entry $(sid, pk_1^{(1)}, v_1 + p_0 + p_1, pk_{\mathcal{F},1}^0, T_1)$ send (swp, ok) to U_0 and U_1.
- Upon receiving $(\mathsf{swp}, sid, pk_0^{(1)})$ from U_1 at time t,
  1. If $b_0^0 = 1 \wedge b_1^0 = 0$, invoke subroutine $\mathsf{Transfer}(id, pk_{\mathcal{F},0}^0, pk_0^{(1)}, v_1 + p_1)$; otherwise abort.
  2. Upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_0}$, delete $(sid, pk_0^{(0)}, v_0 + p_1, pk_{\mathcal{F},0}^0, T_0)$, set $b_0^0 = 0$, and send (swp, ok) to U_0 and U_1.

Fig. 18: Ideal Functionality $\mathcal{F}_{\mathsf{RHSwap}}$ (Part II)

Freeze$(sid, pk, pk_{\mathcal{F}}, v, [T])$: if T is provided, set timeout T; Transfer coins v from account pk to $pk_{\mathcal{F}}$ (controlled by $\mathcal{F}$) via a transaction tx_{lock} and invoking interface $\mathsf{Confirm}(tx_{lock})$ of blockchain functionality $\mathcal{F}_B$. If tx_{lock} has been confirmed by $\mathcal{F}_B$, then respond $(\mathsf{confirmed}, sid)$.

Transfer$(sid, pk_{\mathcal{F}}, pk, v)$: transfer frozen coins v from account $pk_{\mathcal{F}}$ to pk by a transaction tx_{swp} and invoking interface $\mathsf{Confirm}(tx_{swp})$ of blockchain functionality $\mathcal{F}_B$. If tx_{swp} has been confirmed by $\mathcal{F}_B$, then respond $(\mathsf{confirmed}, sid)$.

Unfreeze$(sid, pk_{\mathcal{F}}, pk, v)$: transfer frozen coins v from account $pk_{\mathcal{F}}$ to pk by a transaction tx_{rfd} and invoking interface $\mathsf{Confirm}(tx_{rfd})$ of blockchain functionality $\mathcal{F}_B$. If tx_{rfd} has been confirmed by $\mathcal{F}_B$, then respond $(\mathsf{confirmed}, sid)$.

Fig. 19: The subroutines of ideal functionality $\mathcal{F}_{\mathsf{HSwap}}$ and $\mathcal{F}_{\mathsf{RHSwap}}$

signatures is simulated using the 2PC simulators $\mathcal{S}_{\mathsf{SIG}}^{2PC}$ for the corrupted user;

Hybrid $\mathcal{H}_2$: It is the same as the above execution except that the 2PC protocol Π_{AS} in the premium setup phase to generate pre-signatures is simulated using the 2PC simulators $\mathcal{S}_{\mathsf{AS}}^{2PC}$ for the corrupted user;

Hybrid $\mathcal{H}_3$: It is the same as the above execution except that the adversary corrupts user U_1 and outputs a valid swap transaction $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$ before the simulator initiates swap operation on behalf of U_0, the simulator aborts;

Hybrid $\mathcal{H}_4$: It is the same as the above execution except that the adversary corrupts user U_0 and outputs a valid pre-refund transaction $(\overline{tx}_{rfd}^{(0)}, \overline{\sigma}_{rfd}^{(0)})$ or a valid swap transaction $(tx_{swp}^{(0)}, \sigma_{swp}^{(0)})$ before the timeout T_1. The simulator outputs $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$ and if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{01}, tx_{swp}^{(1)}, \sigma_{swp}^{(1)}) \neq 1$, the simulator aborts;

Hybrid $\mathcal{H}_5$: It is the same as the above execution except that the adversarial U_0 outputs a valid refund transaction $(tx_{rfd}^{(0)}, \sigma_{rfd}^{(0)})$ before timeout T_0, the simulator aborts;

Hybrid $\mathcal{H}_6$: It is the same as the above execution except that the adversarial U_1 outputs a valid refund transaction $(tx_{rfd}^{(1)}, \sigma_{rfd}^{(1)})$ before timeout T_1, the simulator aborts;

Hybrid $\mathcal{H}_7$: It is the same as the above execution except that the adversary corrupts user U_0 and the simulator obtains $(tx_{rfd}^{(1)}, \sigma_{rfd}^{(1)})$ after timeout T_1, if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{10}, tx_{rfd}^{(1)}, \sigma_{rfd}^{(1)}) \neq 1$, the simulator aborts;

Hybrid $\mathcal{H}_8$: It is the same as the above execution except that the adversary corrupts user U_1 and the simulator obtains $(tx_{rfd}^{(0)}, \sigma_{rfd}^{(0)})$ after timeout T_0, if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{01}, tx_{rfd}^{(0)}, \sigma_{rfd}^{(0)}) \neq 1$, the simulator aborts;

Simulator $\mathcal{S}$: The simulator $\mathcal{S}$ is defined as the execution in $\mathcal{H}_8$ while interacting with the ideal functionality $\mathcal{F}$. It simulates the view of the adversary and receives messages from the ideal functionality $\mathcal{F}$.

Below we show the indistinguishability between $\mathcal{H}_0$ and $\mathcal{H}_8$. In addition, we use $\approx_c$ to denote computational indistinguishability for a PPT algorithm.

Hybrid $\mathcal{H}_0 \approx_c \mathcal{H}_1$: The indistinguishability directly follows from the security of 2PC protocol Π_{SIG}. The security of 2PC protocol Π_{SIG} for signature generation guarantees the existence of $\mathcal{S}_{\mathsf{SIG}}^{2PC}$;

Hybrid $\mathcal{H}_1 \approx_c \mathcal{H}_2$: The indistinguishability directly follows from the security of 2PC protocol Π_{AS}. The security of 2PC protocol Π_{AS} for signature generation guarantees the existence of $\mathcal{S}_{\mathsf{AS}}^{2PC}$;

Hybrid $\mathcal{H}_2 \approx_c \mathcal{H}_3$: The only difference between the hybrids is that in $\mathcal{H}_3$ the simulator aborts, if the adversary corrupts user U_1 and outputs a valid swap transaction $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$ before the simulator initiate a swap on behalf of user U_0. With the unforgeability of adaptor signature, the probability of the event triggered in $\mathcal{H}_3$ is negligible;

Hybrid $\mathcal{H}_3 \approx_c \mathcal{H}_4$: The only difference between the hybrids is that in $\mathcal{H}_4$ the simulator aborts, if the adversary corrupts user U_0 and outputs a valid pre-refund transaction $(\overline{tx}_{rfd}^{(0)}, \overline{\sigma}_{rfd}^{(0)})$ or a valid swap transaction $(tx_{swp}^{(0)}, \sigma_{swp}^{(0)})$ before the timeout T_1, while the simulator cannot obtains a valid $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$; With the pre-signature adaptability and witness extractability of adaptor signature, the probability of the event triggered in $\mathcal{H}_4$ is negligible;

Hybrid $\mathcal{H}_4 \approx_c \mathcal{H}_5$: The only difference between the hybrids is that in $\mathcal{H}_5$ the simulator aborts, if the adversarial U_0 outputs a valid refund transaction $(tx_{rfd}^{(0)}, \sigma_{rfd}^{(0)})$ before timeout T_0. With the privacy of VTD, the probability of the event triggered in $\mathcal{H}_5$ is negligible;

Hybrid $\mathcal{H}_5 \approx_c \mathcal{H}_6$: The only difference between the hybrids is that in $\mathcal{H}_6$ the simulator aborts, if adversarial U_1 outputs a valid refund transaction $(tx_{rfd}^{(1)}, \sigma_{rfd}^{(1)})$ before timeout T_1. With the privacy of VTD, the probability of the event triggered in $\mathcal{H}_6$ is negligible;

Hybrid $\mathcal{H}_6 \approx_c \mathcal{H}_7$: The only difference between the hybrids is that in $\mathcal{H}_7$ the simulator aborts, if it cannot obtain a valid $(tx_{rfd}^{(1)}, \sigma_{rfd}^{(1)})$ after timeout T_1; With the soundness of VTD, the probability of the event triggered in $\mathcal{H}_7$ is negligible;

Hybrid $\mathcal{H}_7 \approx_c \mathcal{H}_8$: The only difference between the hybrids is that in $\mathcal{H}_7$ the simulator aborts, if it cannot obtains a valid $(tx_{rfd}^{(0)}, \sigma_{rfd}^{(0)})$ after timeout T_0. With the soundness of VTD, the probability of the event triggered in $\mathcal{H}_8$ is negligible. □

APPENDIX D
PROOF OF THEOREM 2

Proof. We now prove that our round-based HedgeSwap protocol UC-realizes the ideal functionality $\mathcal{F}_{\text{RHSwap}}$. We begin with the real world protocol execution, gradually change the simulation in these hybrids and then we argue about the proximity of neighbouring experiments.

Hybrid $\mathcal{H}_0$: It is the same as the real world protocol execution;

Hybrid $\mathcal{H}_1$: It is the same as the above execution except that the 2PC protocol Π_{SIG} in the premium setup phase to generate signatures is simulated using the 2PC simulators $\mathcal{S}_{\mathsf{SIG}}^{2PC}$ for the corrupted user;

Hybrid $\mathcal{H}_2$: It is the same as the above execution except that the 2PC protocol Π_{AS} in the premium setup phase to generate pre-signatures is simulated using the 2PC simulators $\mathcal{S}_{\mathsf{AS}}^{2PC}$ for the corrupted user;

Hybrid $\mathcal{H}_3$: It is the same as the above execution except that the adversary corrupts user U_1 and outputs a valid swap transaction $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$ before the simulator initiates swap operation on behalf of U_0, the simulator aborts;

Hybrid $\mathcal{H}_4$: It is the same as the above execution except that the adversary corrupts user U_0 and outputs a valid swap transaction $(tx_{swp}^{(0)}, \sigma_{swp}^{(0)})$ before T_1^0. The simulator outputs $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$ and if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{01}^0, tx_{swp}^{(1)}, \sigma_{swp}^{(1)}) \neq 1$, the simulator aborts;

Hybrid $\mathcal{H}_5$: It is the same as the above execution except that the adversarial U_0 outputs a valid refund transaction $(tx_{rfd}^{(0),i}, \sigma_{rfd}^{(0),i})$ before timeout T_0^i for $i \in [0, N]$, the simulator aborts;

Hybrid $\mathcal{H}_6$: It is the same as the above execution except that the adversarial U_1 outputs a valid refund transaction $(tx_{rfd}^{(1),i}, \sigma_{rfd}^{(1)})$ before timeout T_1^i for $i \in [0, N]$, the simulator aborts;

Hybrid $\mathcal{H}_7$: It is the same as the above execution except that the adversary corrupts user U_0 and the simulator obtains $(tx_{rfd}^{(1),i}, \sigma_{rfd}^{(1),i})$ after T_1^i for $i \in [0, N]$, if any $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{(1-j)j}^i, tx_{rfd}^{(1),i}, \sigma_{rfd}^{(1),i}) \neq 1$, the simulator aborts;

Hybrid $\mathcal{H}_8$: It is the same as the above execution except that the adversary corrupts user U_1 and the simulator obtains $(tx_{rfd}^{(0),i}, \sigma_{rfd}^{(0),i})$ after T_0^i for $i \in [0, N]$, if any $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{j(1-j)}^i, tx_{rfd}^{(0),i}, \sigma_{rfd}^{(0),i}) \neq 1$ for $i \in [0, N]$, the simulator aborts;

Simulator $\mathcal{S}$: The simulator $\mathcal{S}$ is defined as the execution in $\mathcal{H}_8$ while interacting with the ideal functionality $\mathcal{F}_{\text{RHSwap}}$. It simulates the view of the adversary and receives messages from the ideal functionality $\mathcal{F}_{\text{RHSwap}}$.

Below we show the indistinguishability between $\mathcal{H}_0$ and $\mathcal{H}_8$.

Hybrid $\mathcal{H}_0 \approx_c \mathcal{H}_1$: The indistinguishability directly follows from the security of 2PC protocol Π_{SIG}. The security of 2PC protocol Π_{SIG} for signature generation guarantees the existence of $\mathcal{S}_{\mathsf{SIG}}^{2PC}$;

Hybrid $\mathcal{H}_1 \approx_c \mathcal{H}_2$: The indistinguishability directly follows from the security of 2PC protocol Π_{AS}. The security of 2PC protocol Π_{AS} for pre-signature generation guarantees the existence of $\mathcal{S}_{\mathsf{AS}}^{2PC}$;

Hybrid $\mathcal{H}_2 \approx_c \mathcal{H}_3$: The only difference between the hybrids is that in $\mathcal{H}_3$ the simulator aborts, if the adversary corrupts user U_1 and outputs a valid swap transaction $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$ before the simulator initiate a swap on behalf of user U_0. With the unforgeability of adaptor signature, the probability of the event triggered in $\mathcal{H}_3$ is negligible;

Hybrid $\mathcal{H}_3 \approx_c \mathcal{H}_4$: The only difference between the hybrids is that in $\mathcal{H}_4$ the simulator aborts, if the adversary corrupts user U_0 and outputs a valid swap transaction $(tx_{swp}^{(0)}, \sigma_{swp}^{(0)})$ before the timeout T_1, while the simulator cannot obtains a valid $(tx_{swp}^{(1)}, \sigma_{swp}^{(1)})$; With the pre-signature adaptability and witness extractability of adaptor signature, the probability of the event triggered in $\mathcal{H}_4$ is negligible;

Hybrid $\mathcal{H}_4 \approx_c \mathcal{H}_5$: The only difference between the hybrids is that in $\mathcal{H}_5$ the simulator aborts, if the adversarial U_0 outputs a valid refund transaction $(tx_{rfd}^{(0),i}, \sigma_{rfd}^{(0),i})$ before timeout T_0^i for

$i \in [0, N]$. With the privacy of VTD, the probability of the event triggered in $\mathcal{H}_5$ is negligible;

Hybrid $\mathcal{H}_5 \approx_c \mathcal{H}_6$: The only difference between the hybrids is that in $\mathcal{H}_6$ the simulator aborts, if adversarial U_1 outputs a valid refund transaction $(tx_{rfd}^{(1),i}, \sigma_{rfd}^{(1),i})$ before timeout T_1^i for $i \in [0, N]$. With the privacy of VTD, the probability of the event triggered in $\mathcal{H}_6$ is negligible;

Hybrid $\mathcal{H}_6 \approx_c \mathcal{H}_7$: The only difference between the hybrids is that in $\mathcal{H}_7$ the simulator aborts, if it cannot obtain a valid $(tx_{rfd}^{(1),i}, \sigma_{rfd}^{(1),i})$ after timeout T_1^i for $i \in [0, N]$; With the soundness of VTD, the probability of the event triggered in $\mathcal{H}_7$ is negligible;

Hybrid $\mathcal{H}_7 \approx_c \mathcal{H}_8$: The only difference between the hybrids is that in $\mathcal{H}_7$ the simulator aborts, if it cannot obtains a valid $(tx_{rfd}^{(0),i}, \sigma_{rfd}^{(0),i})$ after timeout T_0^i for $i \in [0, N]$. With the soundness of VTD, the probability of the event triggered in $\mathcal{H}_8$ is negligible.

□

APPENDIX E
GAME THEORETIC ANALYSIS FOR ATOMIC SWAP

In this section, we use game theoretic to prove that, in a universal atomic swap with timeout interval at least $2\delta + \varphi$, no rational user can increase its utility by deviating from the protocol.

Model. We model the universal swap protocol as a finite extensive-form game with perfect information and chance moves (to capture blockchain confirmation and order uncertainty). We characterize a subgame-perfect Nash equilibrium (SPNE) by backward induction.

The game involves two rational players U_0 and U_1. User U_0 locks coins v_0 on B_0 with timeout T_0, and user U_1 locks coins v_1 on B_1 with T_1, where $T_0 - T_1 \geq 2\delta + \varphi$. We adopt the same network assumptions as in [7], [19] (see Section III). We denote by $u_j(\cdot)$ user U_j's utility, defined as the sum of the values of the assets that U_j holds at the end of the game. Each user is assumed to prefer completing the swap to refunding it, i.e., $u_0(v_1) > u_0(v_0)$ and $u_1(v_0) > u_1(v_1)$; otherwise, it would not agree to the swap in the first place. If both redeem and refund transactions for the same coins are broadcast simultaneously (within the network delay bound), we assume that each is included with probability $1/2$. We capture this as a chance node with equal probabilities.

Figure 20 shows the game tree for the universal swap protocol, starting from the point where both users have locked their coins. Intuitively, U_0 chooses whether and when to redeem v_1, and U_1 chooses whether and when to redeem v_0; otherwise, both parties refund after their respective timeouts.

Theorem 3. *The strategy profile in which U_0 redeems v_1 before time T_1, and U_1 redeems v_0 before time T_0, forms a subgame-perfect Nash equilibrium.*

Proof. We proceed by backward induction. Consider the subgame that starts at any history in which U_1 has observed the witness but has not yet decided whether to redeem v_0.

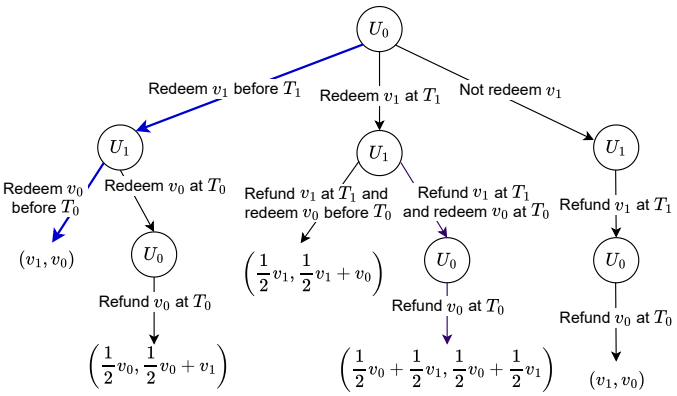


Fig. 20: Game tree. Each terminal node denotes a payoff pair (u_0, u_1), representing the utilities of U_0 and U_1.

If U_1 redeems v_0 before T_1, its utility is $u_1(v_0)$. If instead U_1 delays redeeming until time T_0, its utility is $u_1((1/2)v_0)$. Since $u_1(v_0) > u_1((1/2)v_0)$, in every subgame, redeeming v_0 before T_0 is a best response for U_1.

Given U_1's best response, if U_0 redeems v_1 before T_1, then by the timeout interval $T_0 - T_1 \geq 2\delta + \varphi$ and the network assumptions, U_1 has enough time to observe the witness and to redeem v_0 before T_0. Hence, U_0 obtains utility $u_0(v_1)$. If U_0 chooses to redeems v_1 at time T_1, there is a $1/2$ chance that U_1's refund is confirmed, while the witness becomes observable from U_0's redeem transaction, which enables U_1 to redeem v_0. In this case, U_0 obtains utilities $u_0((1/2)v_1)$ and U_1 obtains $u_1((1/2)v_1 + v_0)$. If U_0 does not redeem v_1, then U_1 refunds v_1 at T_1 and U_0 refunds v_0 at T_0, so U_0 obtains utility $u_0(v_0)$. Since $u_0(v_1) > u_0(v_0) > u_0((1/2)v_1)$, U_0's best response is to redeem v_1 before T_1.

Therefore, the strategy profile in which U_0 redeems v_1 before T_1 and U_1 redeems v_0 before T_0 is an SPNE. □